

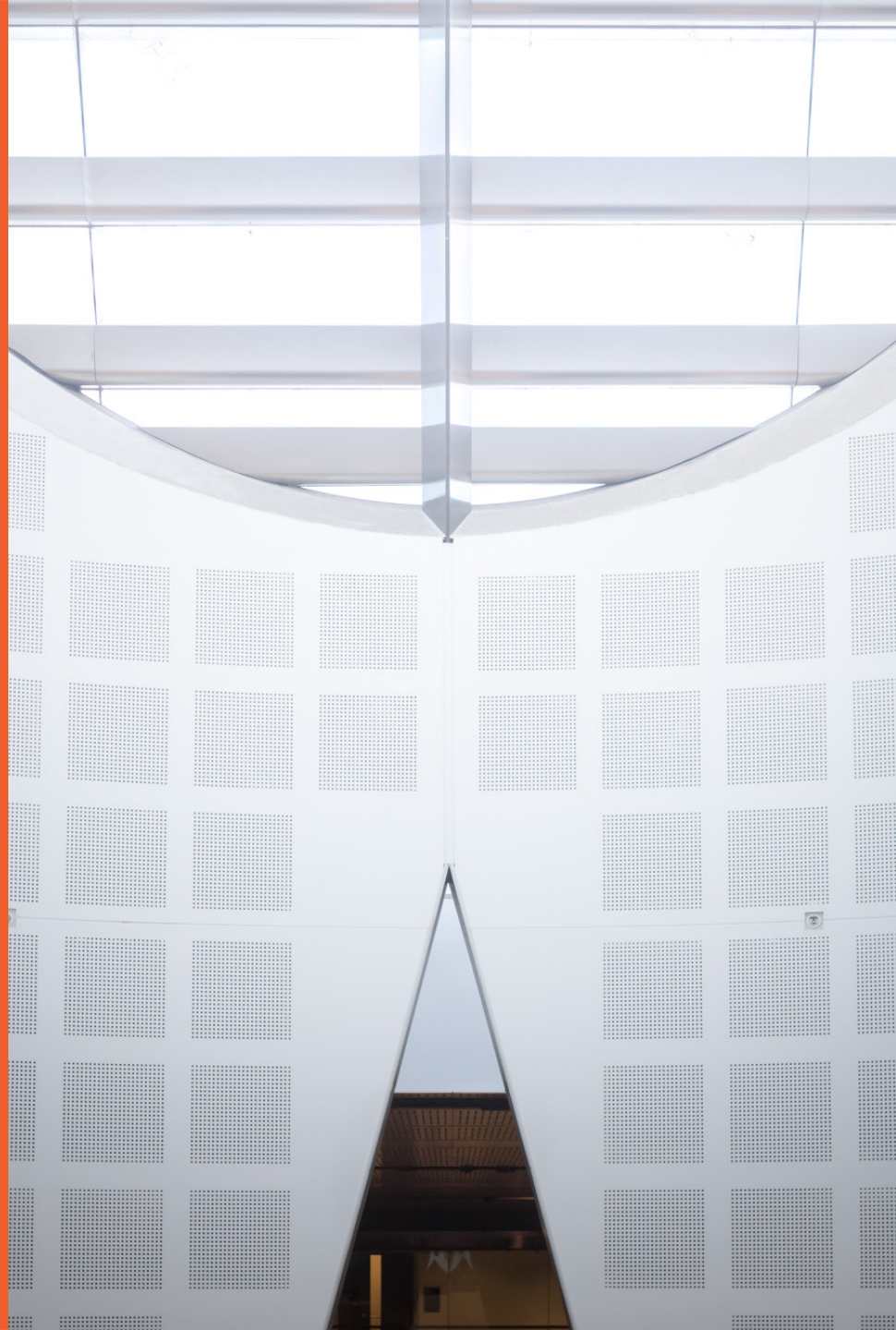
Deep Generative Models

A/Prof Chang Xu

School of Computer Science

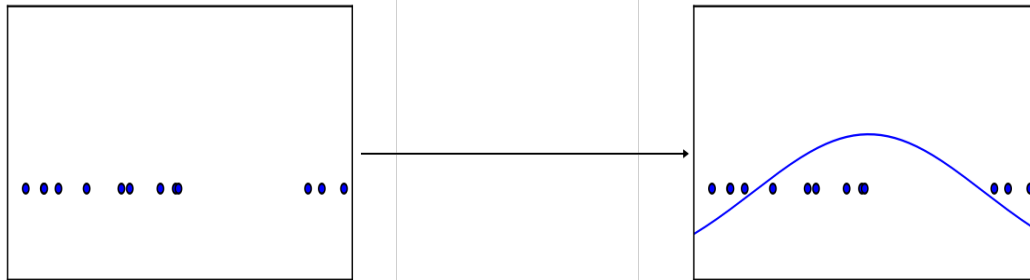


THE UNIVERSITY OF
SYDNEY

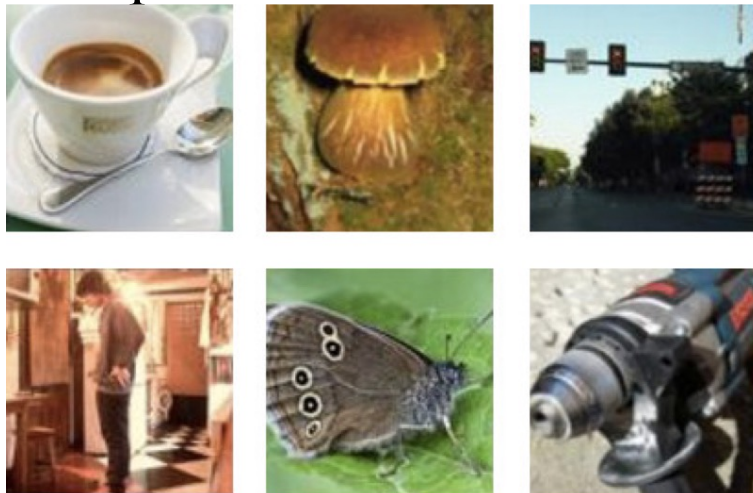


Generative Modeling

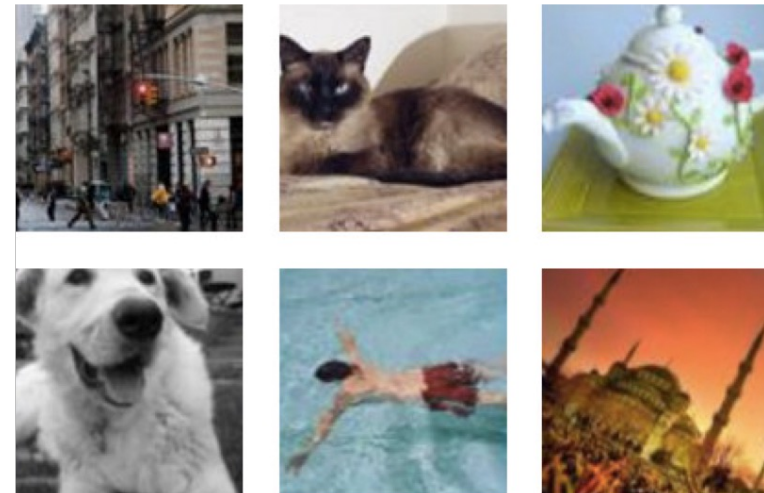
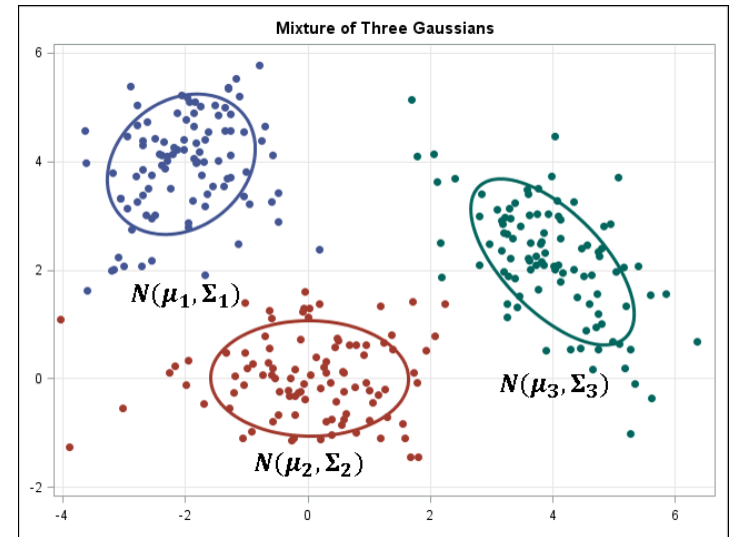
- Density Estimation



- Sample Generation



Training Sample



Generated Sample

Why study generative model?

- Realistic generation tasks

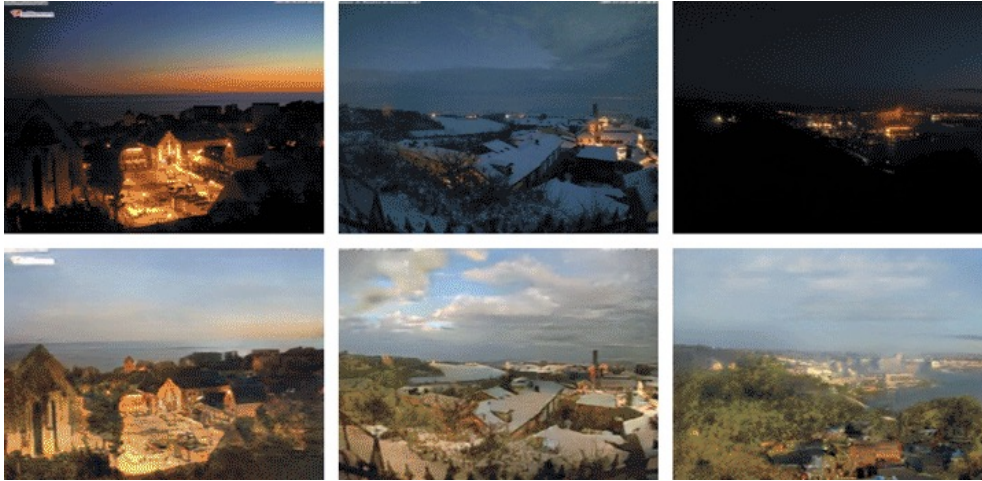


Image credit to [Jun-Yan Zhu et al. 2017]



Image credit to [Phillip Isola et al. 2017]

- Simulate possible futures for planning (e.g. Stock Market Prediction)
- Training generative models can also enable inference of latent representations that can be useful as general features

Generative Adversarial Networks

Generative Adversarial Networks

0.6551
(RNG)



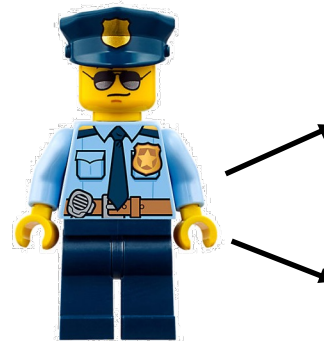
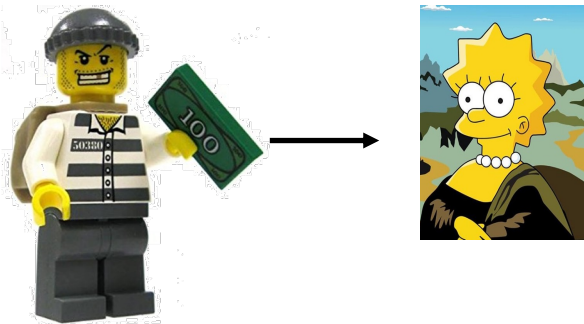
Generative Adversarial Networks

- A **counterfeiter-police game** between two components: a generator **G** and a discriminator **D**
- **G**: counterfeiter, trying to fool police with fake currency
- **D**: police, trying to detect the counterfeit currency
- Competition drives both to improve, until counterfeits are *indistinguishable* from genuine currency



Generative Adversarial Networks

- A **two-player game** between two components: a generator **G** and a discriminator **D**
- **G**: try to fool the discriminator by generating real-looking images
- **D**: try to distinguish between real and fake images



✓



✗

Generative Adversarial Networks

- A **two-player game** between two components: a generator **G** and a discriminator **D**
- **G**: try to fool the discriminator by generating real-looking images
- **D**: try to distinguish between real and fake images

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

D predicting that
real data is genuine

D predicting that
G's generated data is fake

Discriminator outputs likelihood in (0,1) of real image.

Generative Adversarial Networks

- A **two-player game** between two components: a generator **G** and a discriminator **D**
- **G**: try to fool the discriminator by generating real-looking images
- **D**: try to distinguish between real and fake images

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

D predicting that
real data is genuine

D predicting that
G's generated data is fake

D's goal: maximize objective such that $D(\mathbf{x})$ is close to 1 (real) and $D(G(\mathbf{z}))$ is close to 0 (fake)

G's goal: minimize objective such that $D(G(\mathbf{z}))$ is close to 1 (discriminator is fooled into thinking generated $G(\mathbf{z})$ is real)

Generative Adversarial Networks

- A **two-player game** between two components: a generator **G** and a discriminator **D**
- **G**: try to fool the discriminator by generating real-looking images
- **D**: try to distinguish between real and fake images

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

D predicting that
real data is genuine

D predicting that
G's generated data is fake

Q: What can we use to represent D and G?

A: Neural networks!

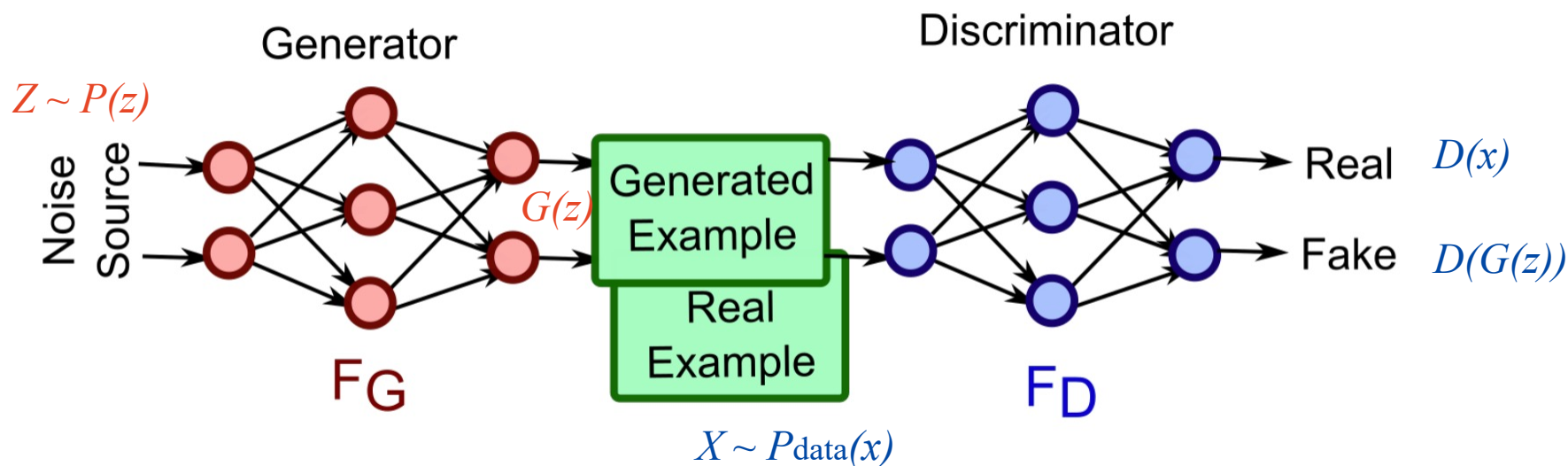
Generative Adversarial Networks

- A **two-player game** between two components: a generator **G** and a discriminator **D**

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

D predicting that
real data is genuine

D predicting that
G's generated data is fake



Generative Adversarial Networks

- A **two-player game** between two components: a generator **G** and a discriminator **D**

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

- Alternate between:

1. Gradient ascent on **D**

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

2. Gradient descent on **G**

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

Algorithm

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Sample minibatch of m examples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\mathbf{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(\mathbf{x}^{(i)}) + \log \left(1 - D(G(\mathbf{z}^{(i)})) \right) \right].$$

end for

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Update the generator by descending its stochastic gradient:

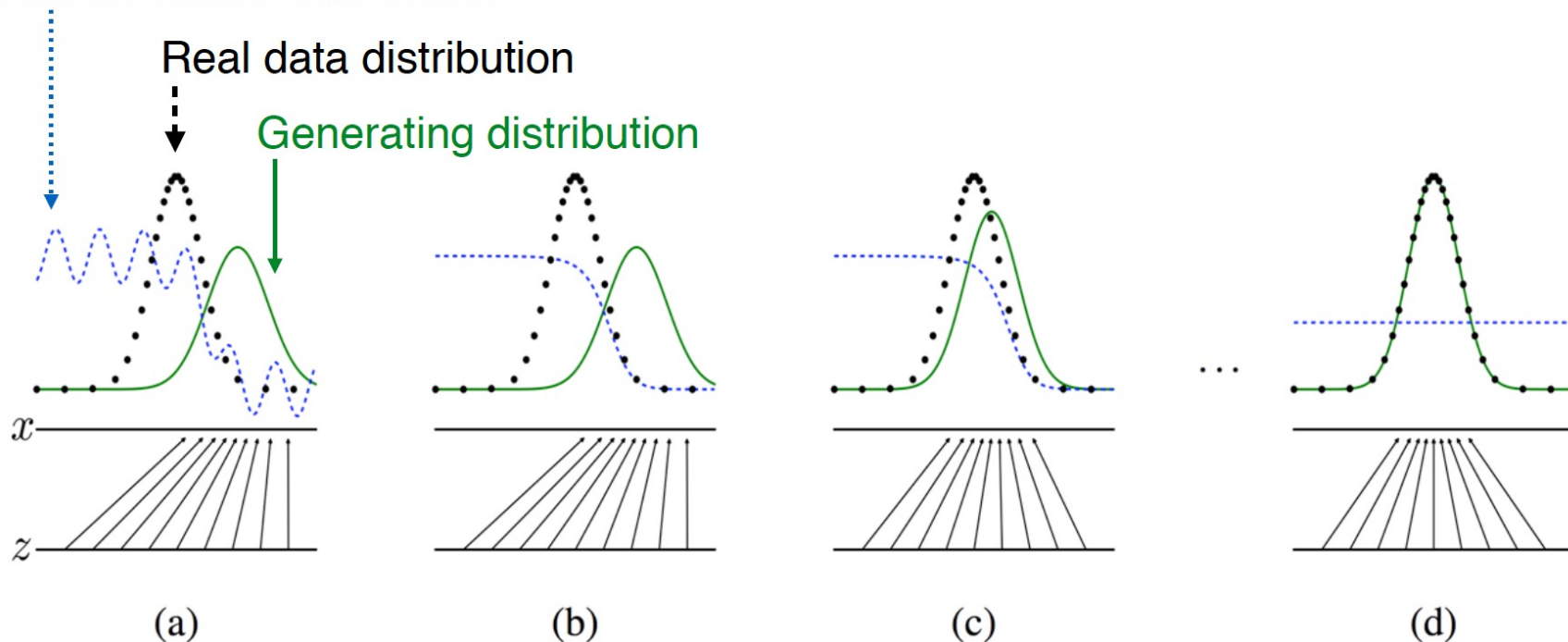
$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log \left(1 - D(G(\mathbf{z}^{(i)})) \right).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

A simple example

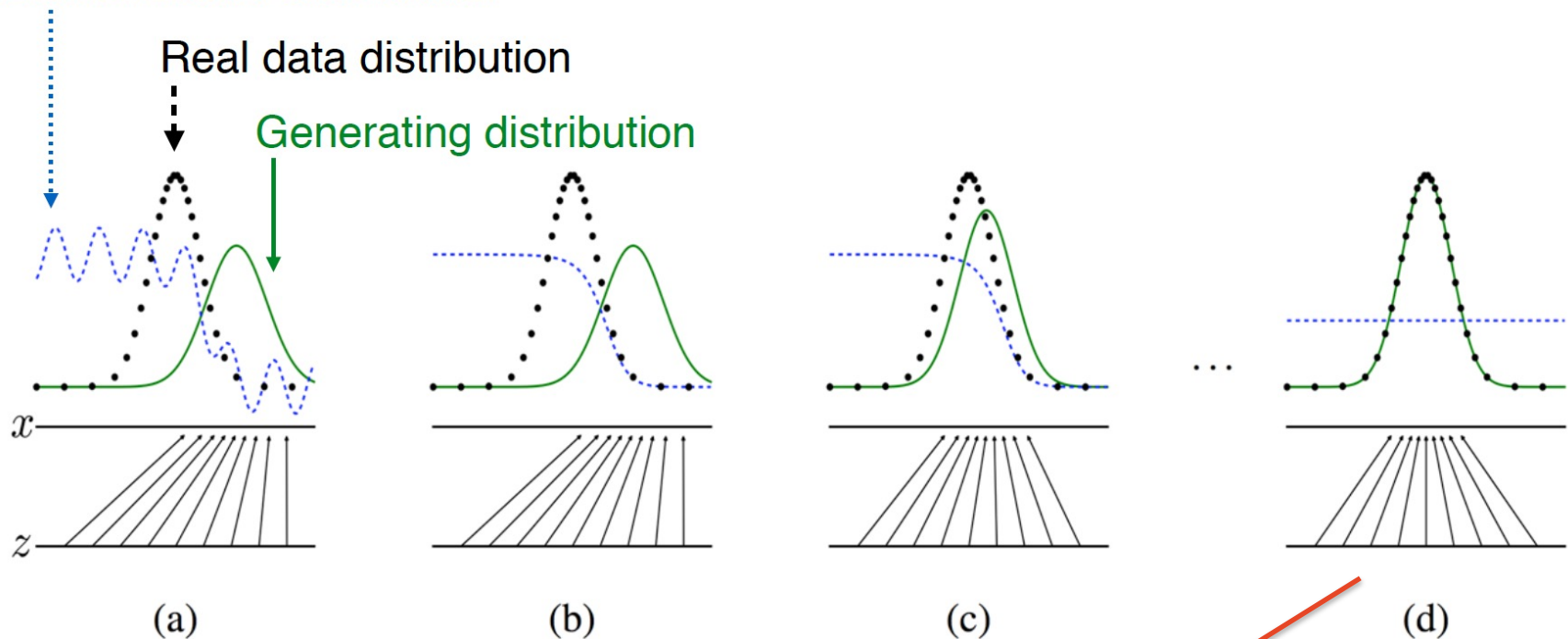
Discriminative distribution



Generative adversarial nets are trained by simultaneously updating the discriminative distribution (blue line) so that it discriminates between samples from the data generating distribution (black, dotted line) p_x from those of the generative distribution p_g (green line). The lower horizontal line is the domain from which z is sampled, in this case uniformly. The horizontal line above is part of the domain of x . The upward arrows show how the mapping $x = G(z)$ imposes the non-uniform distribution p_g on transformed samples. G contracts in regions of high density and expands in regions of low density of p_g .

A simple example

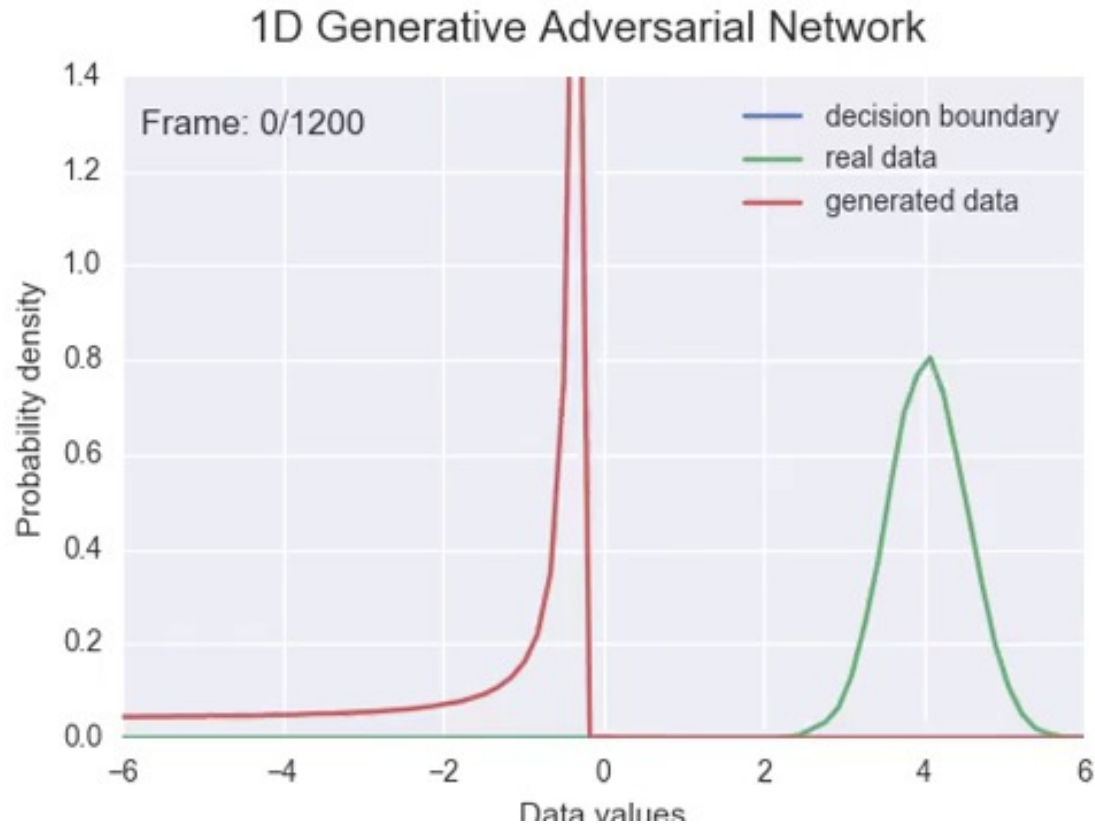
Discriminative distribution



After several steps of training, if G and D have enough capacity, they will reach a point at which both cannot improve because $p_g = p_{data}$. The discriminator is unable to differentiate between the two distributions, i.e. $D(\mathbf{x}) = 1/2$.

A simple example

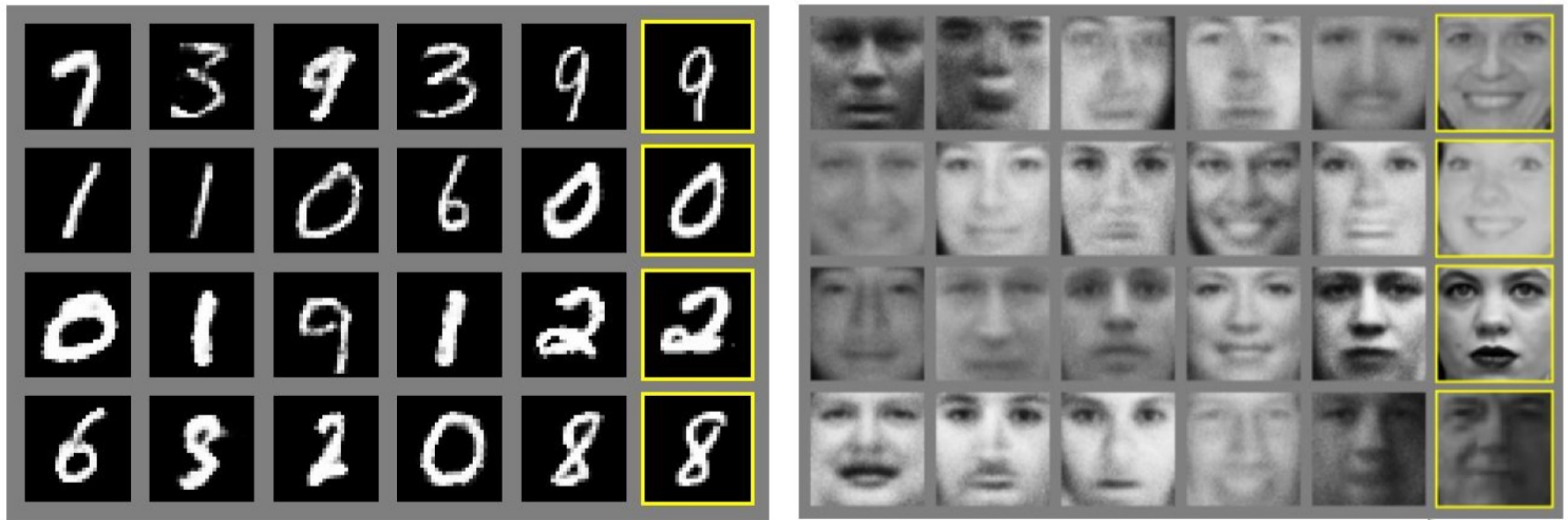
When the data distribution is a 1-D Gaussian distribution



<http://blog.aylien.com/introduction-generative-adversarial-networks-code-tensorflow/>

Generative Adversarial Networks

- Generated samples



Nearest neighbor from training set

Generative Adversarial Networks

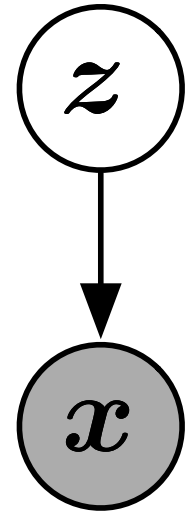
- Generated samples (CIFAR-10)



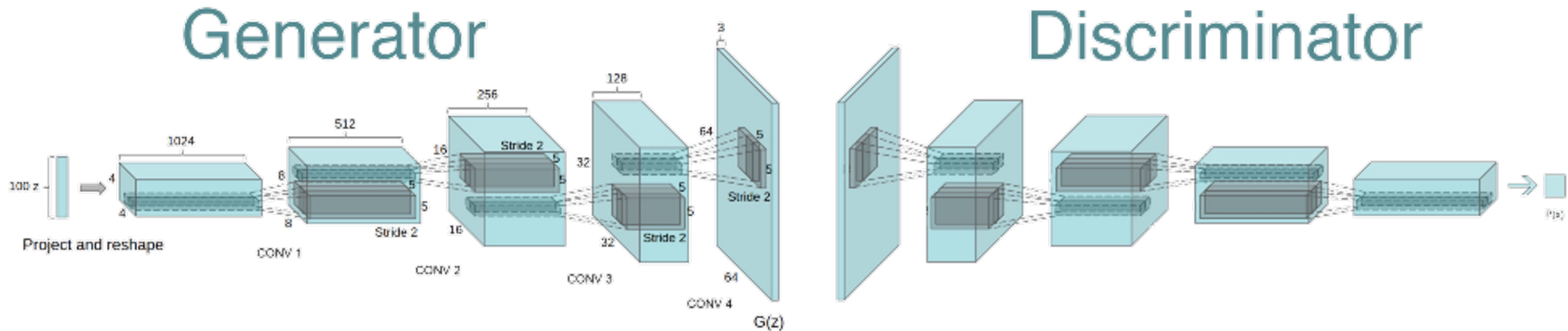
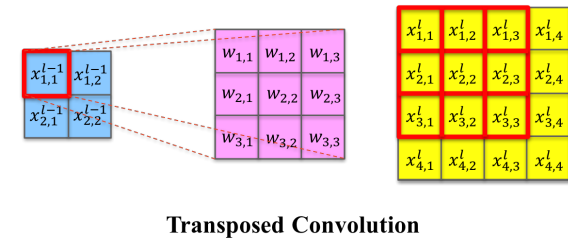
Nearest neighbor from training set

Generative Adversarial Networks

- $x = G(z; \theta^{(G)})$
- It is only required that, G is differentiable
- So, having training data from real data distribution p_r what we want is a generative model that can draw samples from generator's distribution p_g , where $p_r \approx p_g$
- Don't need to write a formula for p_g just learn to draw sample directly.



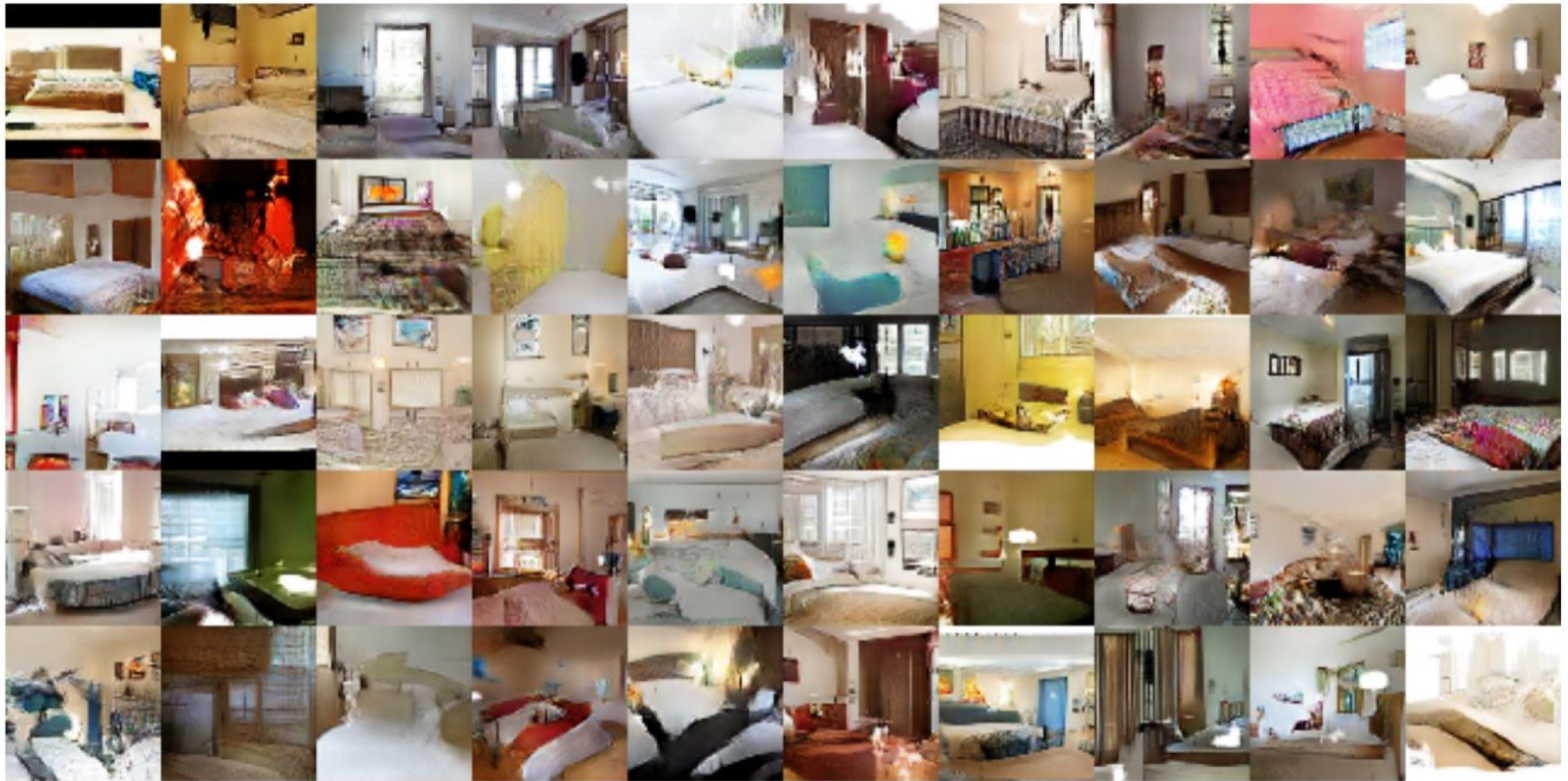
Deep Convolutional GANs (DCGANs)



- Replace any pooling layers with strided convolutions (discriminator) and fractional-strided convolutions (generator).
- Use batchnorm in both the generator and the discriminator.
- Remove fully connected hidden layers for deeper architectures.
- Use ReLU activation in generator for all layers except for the output, which uses Tanh.
- Use LeakyReLU activation in the discriminator for all layers.

Deep Convolutional GANs (DCGANs)

Samples
from the
model look
much
better!

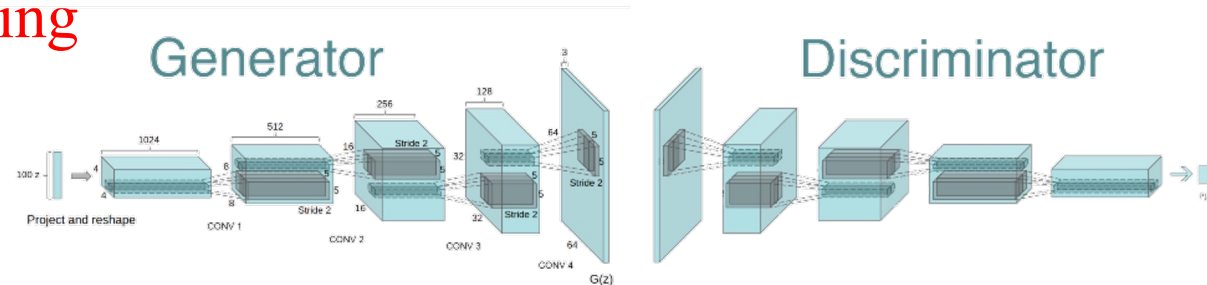


Radford et al, “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks”, ICLR 2016

Wasserstein GAN

Difficulty of Generative Adversarial Networks

- The gradient vanishing
- The full objective



$$J^{(D)} = -E_{x \sim P_r} [\log D(x)] - E_{x \sim P_g} [\log(1 - D(x))]$$

$$J^{(G)} = E_{x \sim P_g} [\log(1 - D(x))]$$

- At the very early phase of training, D is very easy to be confident in detecting G, so D will output almost always 0

In GAN, better discriminator leads to worse vanishing gradient in its generator!

Proof Sketch: [Martin Arjovsky et al., Wasserstein GAN, 2017]

- Minimizing generator yields minimizing the JS divergence when the discriminator is optimal:

$$J^{(D)} = -E_{x \sim P_r} [\log D(x)] - E_{x \sim P_g} [\log(1 - D(x))]$$

- The optimal $\mathbf{D}$ for any P_r and P_g is always:

$$D^*(x) = \frac{P_r(x)}{P_r(x) + P_g(x)} \quad -\frac{P_r(x)}{D(x)} + \frac{P_g(x)}{1 - D(x)} = 0$$

- The generative loss (by adding a term independent of P_g) is:

$$J^{(G)} = E_{x \sim P_g} [\log(1 - D(x))]$$

Plug D^* into $J^{(G)}$:

$$J^{(D^*)} = 2 \textcolor{red}{JS}(P_r || P_g) - 2 \log 2$$

 Jensen-Shannon divergence

So, when $\mathbf{D}$ is *optimal*, minimizing the loss is equal to minimizing the *JS divergence*.

JS divergence:

$$\text{JS}(p \parallel q) = \frac{1}{2} \text{KL}(p \parallel m) + \frac{1}{2} \text{KL}(q \parallel m)$$

where

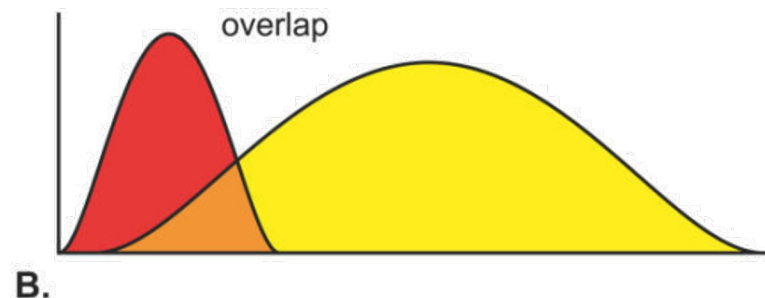
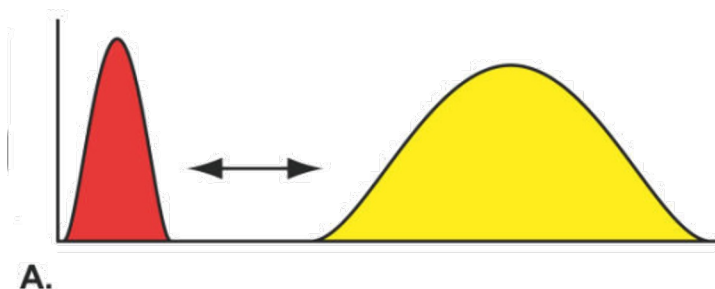
$$m(x) = \frac{1}{2} (p(x) + q(x))$$

$$\text{KL}(p \parallel q) = - \sum_x p(x) \log \left(\frac{q(x)}{p(x)} \right)$$

Proof Sketch:

$$J^{(D^*)} = 2JS(P_r || P_g) - 2 \log 2$$

- If the supports (underlying low-dimension manifolds) of P_r and P_g (almost) have no overlap, then $JS(P_r || P_g) = \log 2$, and thus the gradient of $J^{(G)}$ w.r.t. P_g vanishes.
- The probability that the support of P_r and P_g have almost zero overlap is 1.



Preliminaries: distance measures for distributions

- JS

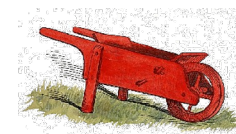
$$JS(P||Q) = \frac{1}{2}KL(P||\frac{P+Q}{2}) + \frac{1}{2}KL(Q||\frac{P+Q}{2})$$

- Wasserstein

$$W(P||Q) = \inf_{\gamma \in \Pi(P,Q)} \mathbb{E}_{(x,y) \sim \gamma} [|x - y|]$$

the set of all joint γ distributions whose marginals are P and Q , respectively.

γ indicates a plan to transport “mass” from x to y , when performing P into Q .



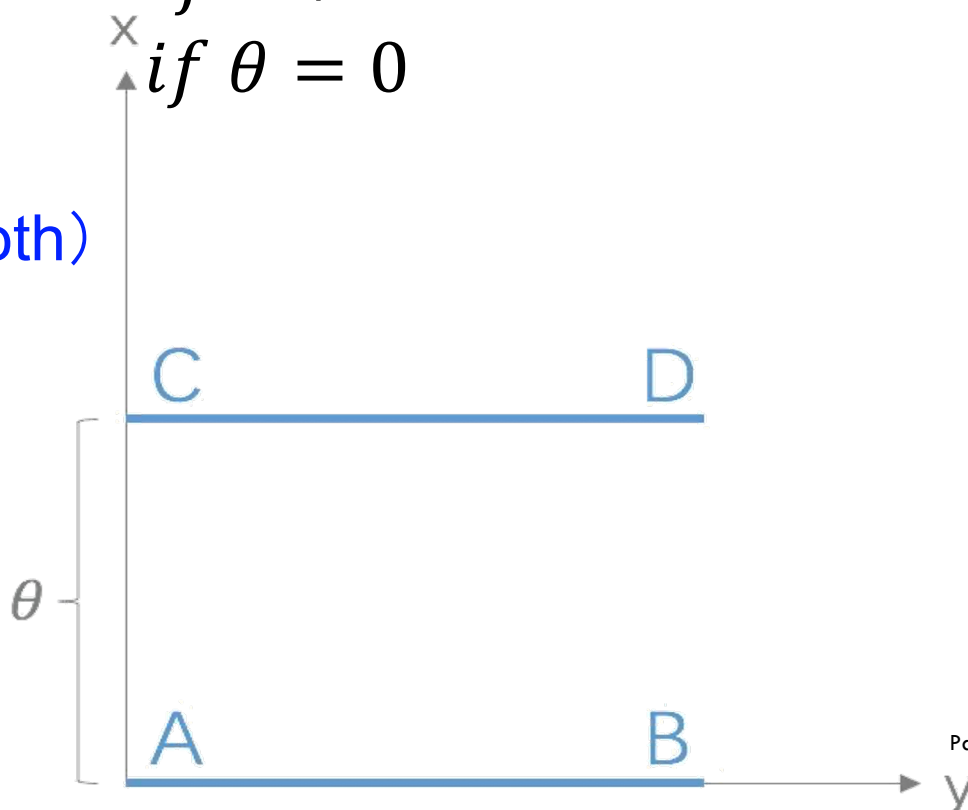
The Wasserstein (or Earth-Mover) distance is then the “cost” of the **optimal** transport plan

Examples

$$KL(P_1 || P_2) = \begin{cases} +\infty & \text{if } \theta \neq 0 \\ 0 & \text{if } \theta = 0 \end{cases}$$

$$JS(P_1 || P_2) = \begin{cases} \log 2 & \text{if } \theta \neq 0 \\ 0 & \text{if } \theta = 0 \end{cases}$$

$$W(P_1, P_2) = |\theta| \text{ (Smooth)}$$



Wasserstein GANs

- The Earth-Mover (EM) distance or Wasserstein-1:

$$W(P||Q) = \inf_{\gamma \in \Pi(P,Q)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|]$$

have nicer properties when optimized than JS

- However, the infimum is highly intractable.
- Wasserstein distance has a duality form

$$\begin{aligned} W(P_r, P_g) &= \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim P_r} [f(x)] - \mathbb{E}_{x \sim P_g} [f(x)] \\ &= \frac{1}{K} \sup_{\|f\|_L \leq K} \mathbb{E}_{x \sim P_r} [f(x)] - \mathbb{E}_{x \sim P_g} [f(x)] \end{aligned}$$

where supremum is over all the K-Lipschitz functions

Wasserstein GANs

- Wasserstein distance has a duality form

$$\begin{aligned} W(P_r, P_g) &= \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim P_r}[f(x)] - \mathbb{E}_{x \sim P_g}[f(x)] \\ &= \frac{1}{K} \sup_{\|f\|_L \leq K} \mathbb{E}_{x \sim P_r}[f(x)] - \mathbb{E}_{x \sim P_g}[f(x)] \end{aligned}$$

where supremum is over all the K-Lipschitz functions

- Consider a w -parameterized family of functions $\{f_w\}_{w \in W}$ that are all K-Lipschitz

$$W(P_r, P_g) = \max_{w \in W} \mathbb{E}_{x \sim P_r}[f_w(x)] - \mathbb{E}_{x \sim P_g}[f_w(x)]$$

For example, $W = [-c, c]^l$. To satisfy this requirement, WGAN enforces the weights of D lie within a compact space $[-c, c]$ by applying **weight clipping**

Wasserstein GANs

- The loss for **discriminator/critic** f_w

$$\mathbb{E}_{x \sim P_r} [f_w(x)] - \mathbb{E}_{x \sim P_g} [f_w(x)]$$

- f_w 's goal: maximize Wasserstein distance between real data distribution and generative distribution

- The loss for **generator**

$$-\mathbb{E}_{x \sim P_g} [f_w(x)] = -\mathbb{E}_{z \sim p(z)} [f_w(g_\theta(z))]$$

- g_θ 's goal: minimize Wasserstein distance between real data distribution and generative distribution

Algorithm

Algorithm 1 WGAN, our proposed algorithm. All experiments in the paper used the default values $\alpha = 0.00005$, $c = 0.01$, $m = 64$, $n_{\text{critic}} = 5$.

Require: : α , the learning rate. c , the clipping parameter. m , the batch size. n_{critic} , the number of iterations of the critic per generator iteration.

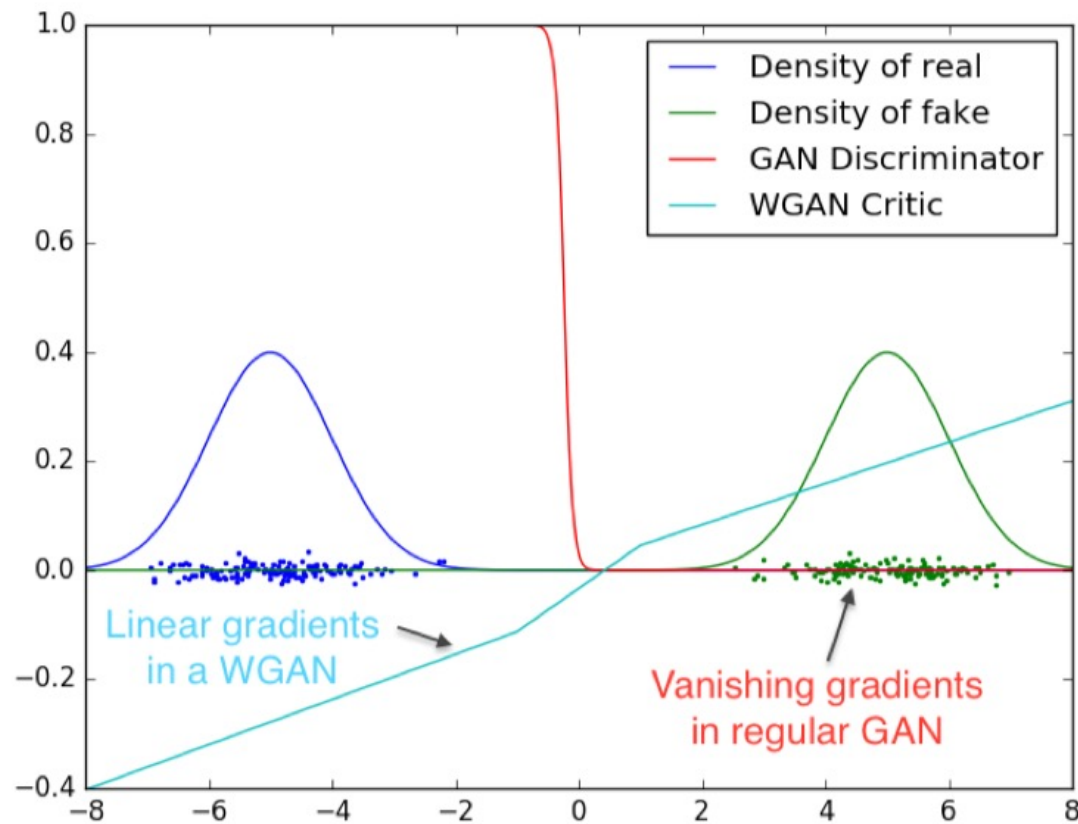
Require: : w_0 , initial critic parameters. θ_0 , initial generator's parameters.

```

1: while  $\theta$  has not converged do
2:   for  $t = 0, \dots, n_{\text{critic}}$  do
3:     Sample  $\{x^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$  a batch from the real data.
4:     Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
5:      $g_w \leftarrow \nabla_w \left[ \frac{1}{m} \sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)})) \right]$  → compute gradient of  $f_w$ 
6:      $w \leftarrow w + \alpha \cdot \text{RMSPProp}(w, g_w)$  → gradient ascent on  $f_w$ 
7:      $w \leftarrow \text{clip}(w, -c, c)$  → weight clipping to satisfy that functions  $\{f_w\}_{w \in W}$  are all K-Lipschitz
8:   end for
9:   Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
10:   $g_\theta \leftarrow -\nabla_\theta \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))$  → compute gradient of  $g_\theta$ 
11:   $\theta \leftarrow \theta - \alpha \cdot \text{RMSPProp}(\theta, g_\theta)$  → gradient decent on  $g_\theta$ 
12: end while

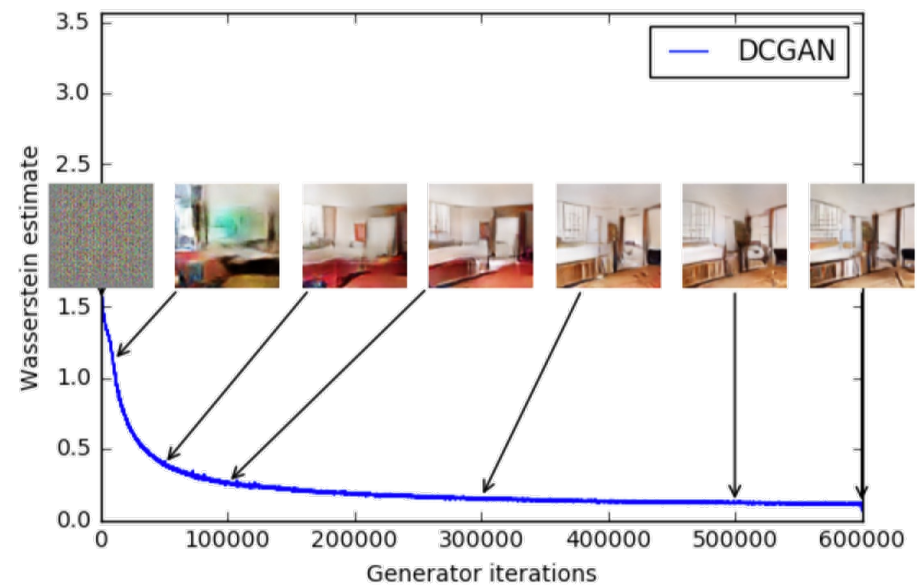
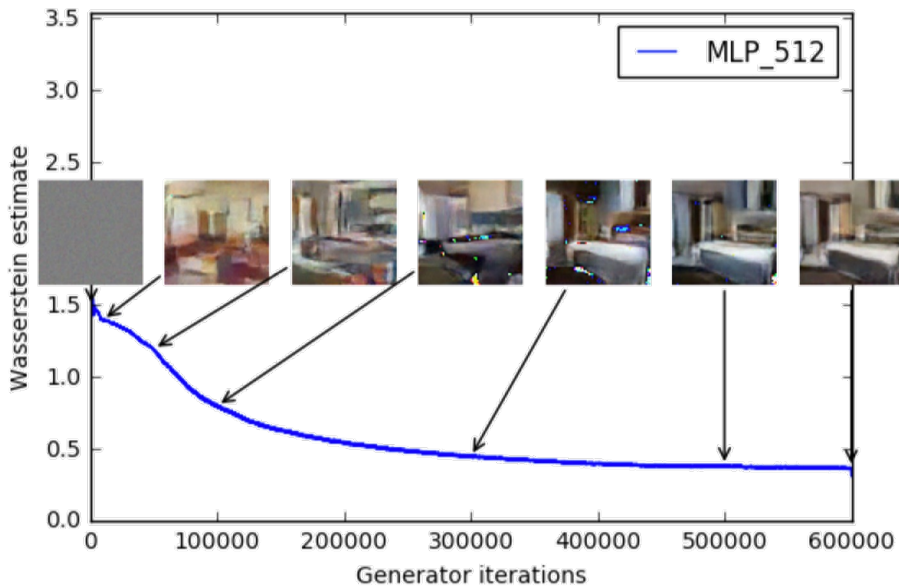
```

Algorithm



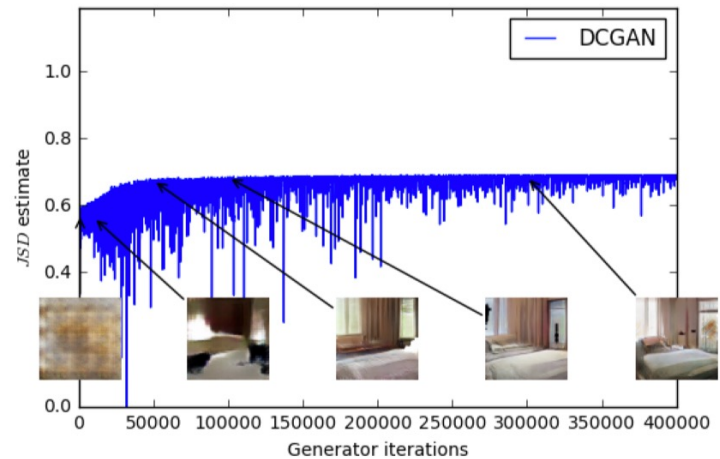
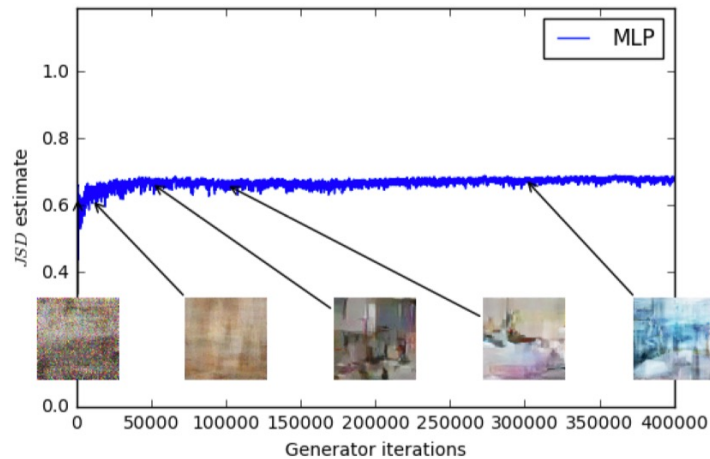
Optimal discriminator and critic when learning to differentiate two Gaussians. As we can see, the discriminator of a minimax GAN saturates and results in vanishing gradients. WGAN critic provides very clean gradients on all parts of the space.

Meaningful loss metric



Meaningful loss metric

– Vanilla GANs



JS estimates for an MLP generator (upper left) and a DCGAN generator (upper right) trained with the standard GAN procedure. Both had a DCGAN discriminator. Both curves have increasing error. Samples get better for the DCGAN but the JS estimate increases or stays constant, pointing towards no significant correlation between sample quality and loss.

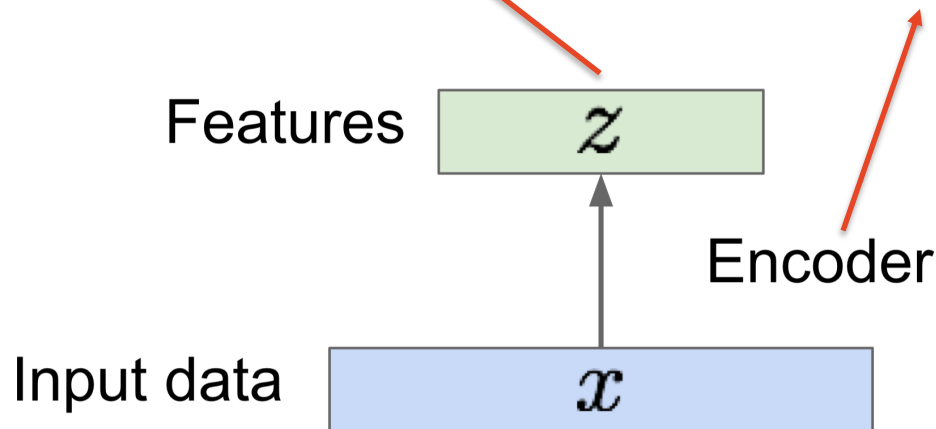
Variational Auto-Encoder

Recap: Autoencoder

- Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data

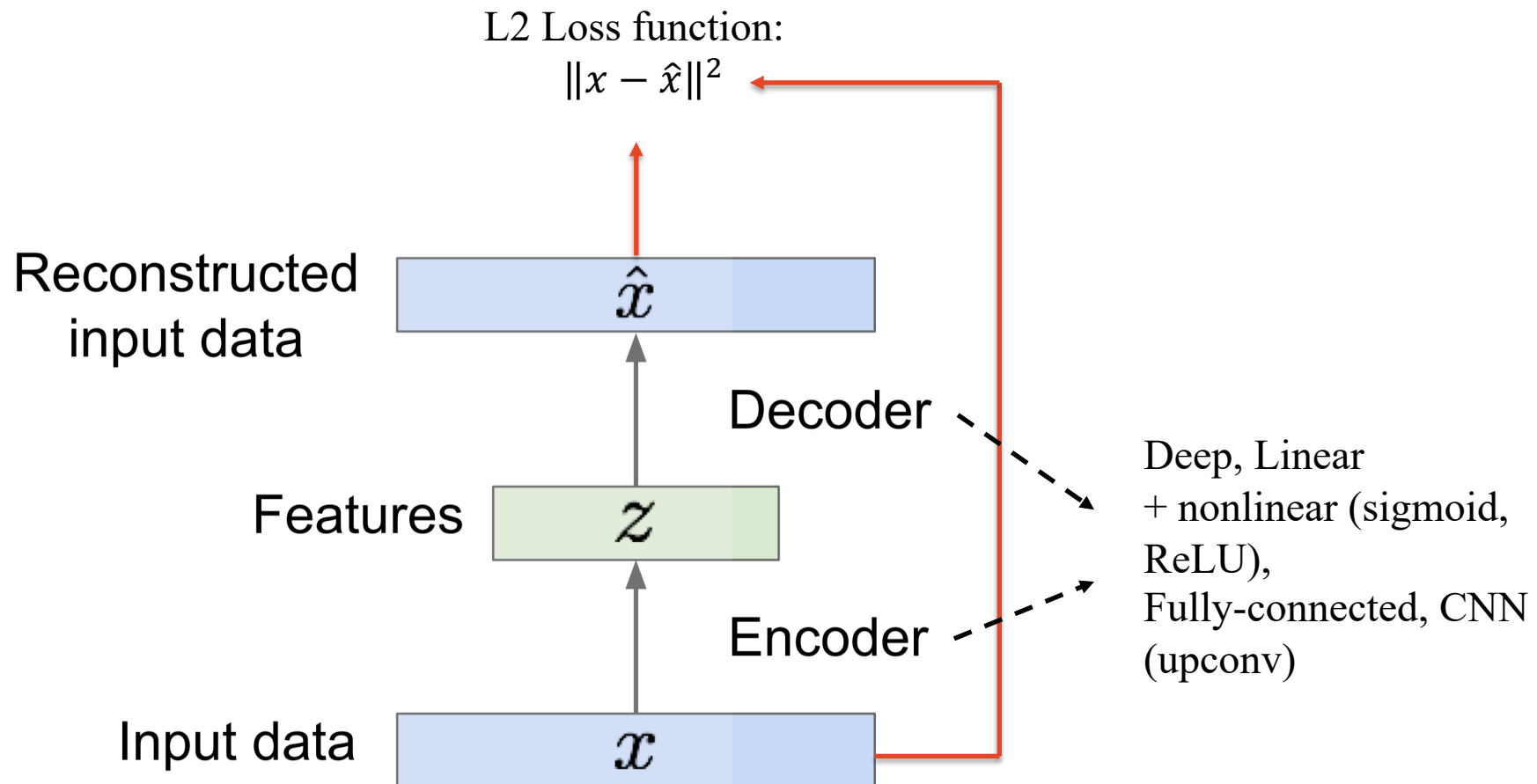
z usually smaller than x
(dimension reduction)

Deep, Linear
+ nonlinear (sigmoid, ReLU),
Fully-connected, CNN

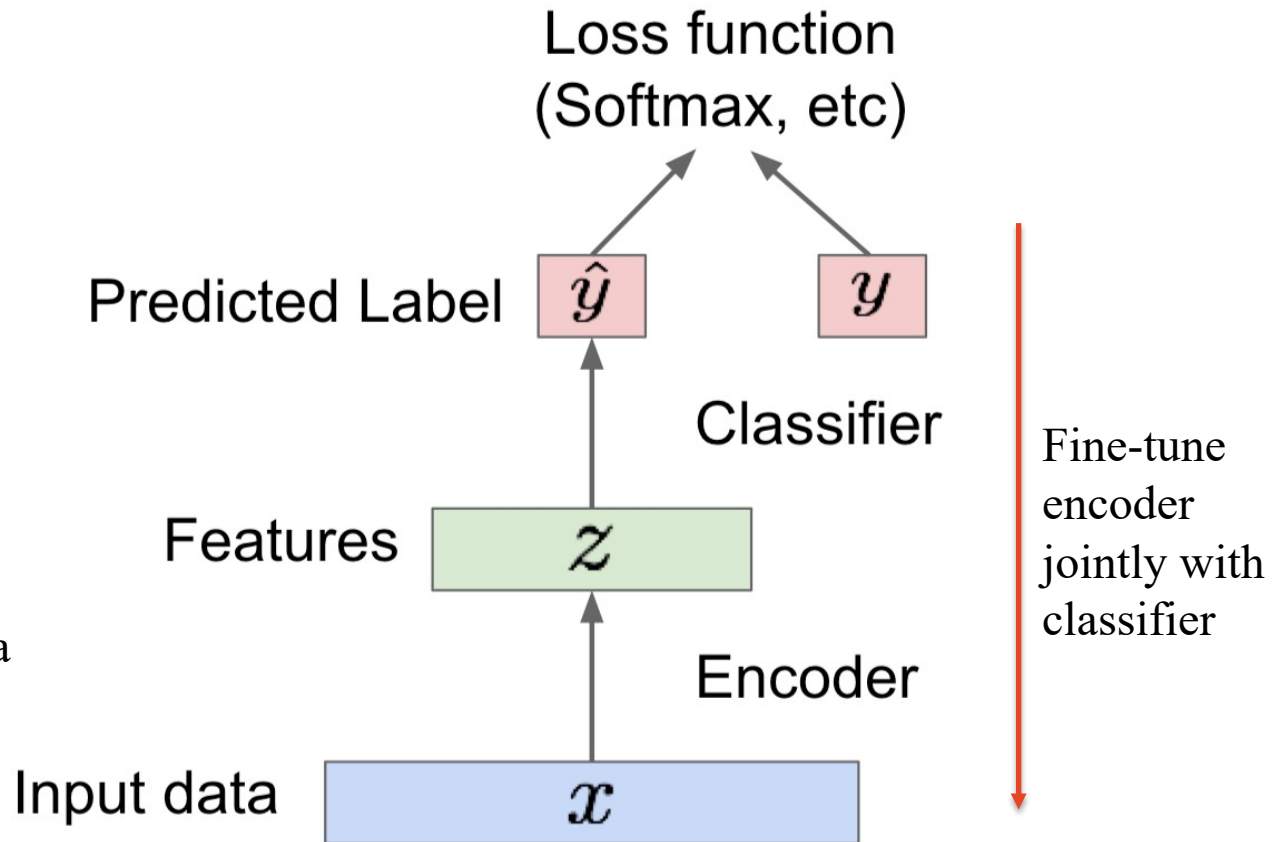


Recap: Autoencoder

- Train such that features can be used to reconstruct original data
“Autoencoding” – encoding itself



Recap: Autoencoder



Throw away
decoder

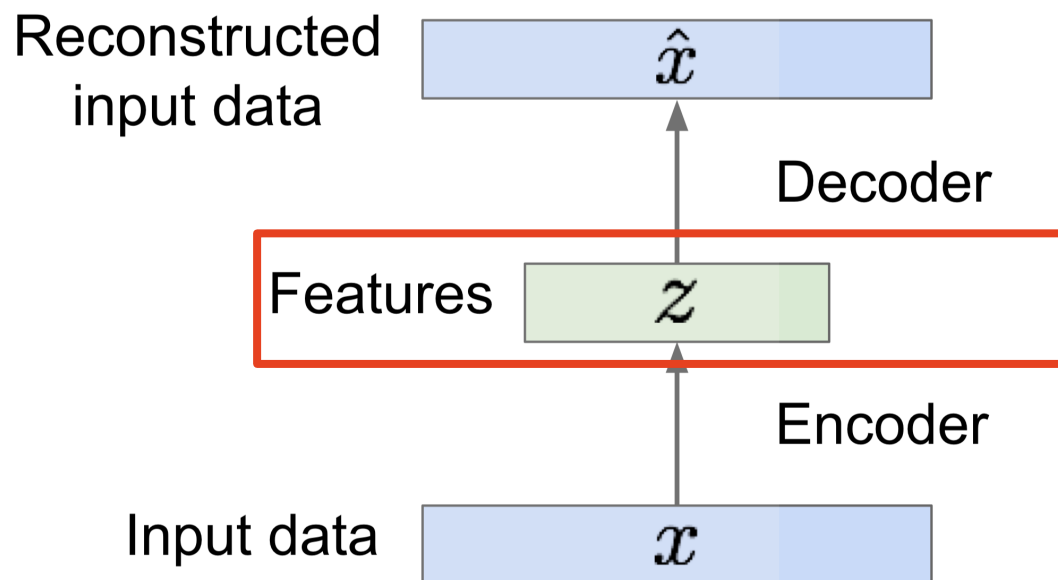
Encoder can be
used to initialize a
supervised model

Fine-tune
encoder
jointly with
classifier

Recap: Autoencoder

Autoencoders can reconstruct data, and can learn features to initialize a supervised model. Features capture factors of variation in training data.

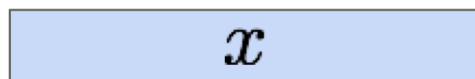
Can we **generate new images** from an autoencoder?



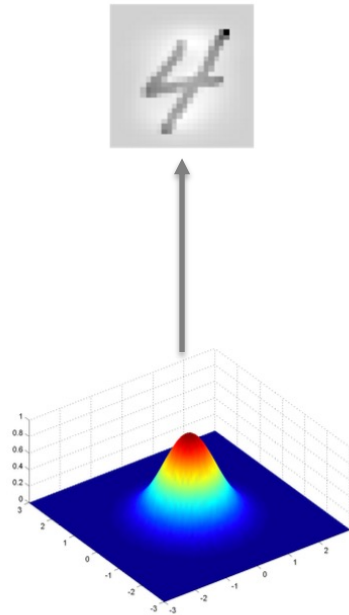
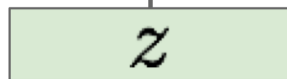
Variational Autoencoders

- Probabilistic spin on autoencoders - will let us sample from the model to generate data!
- Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from underlying unobserved (latent) representation z

Sample from
true conditional
 $p_{\theta^*}(x|z^{(i)})$

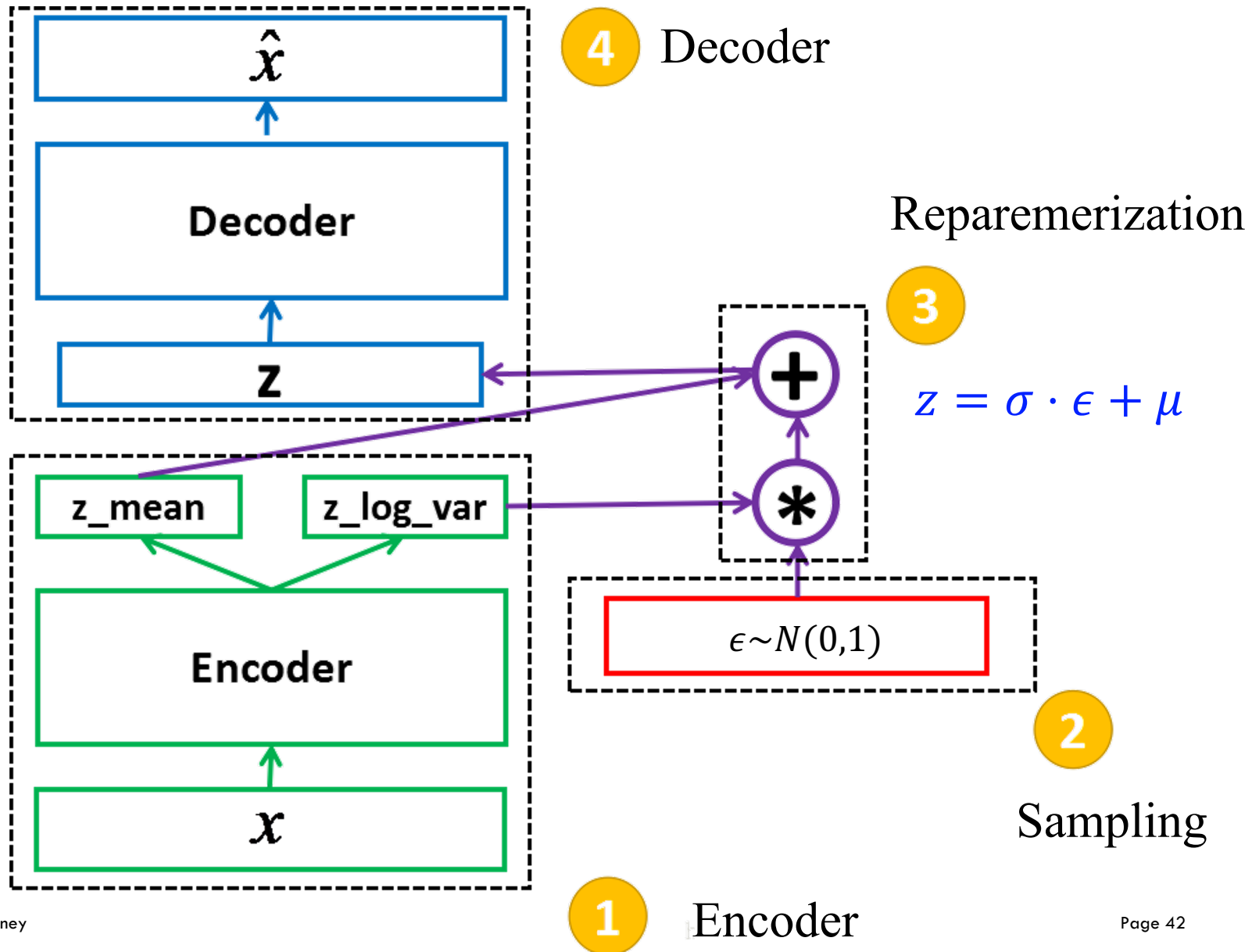


Sample from
true prior $p_{\theta^*}(z)$

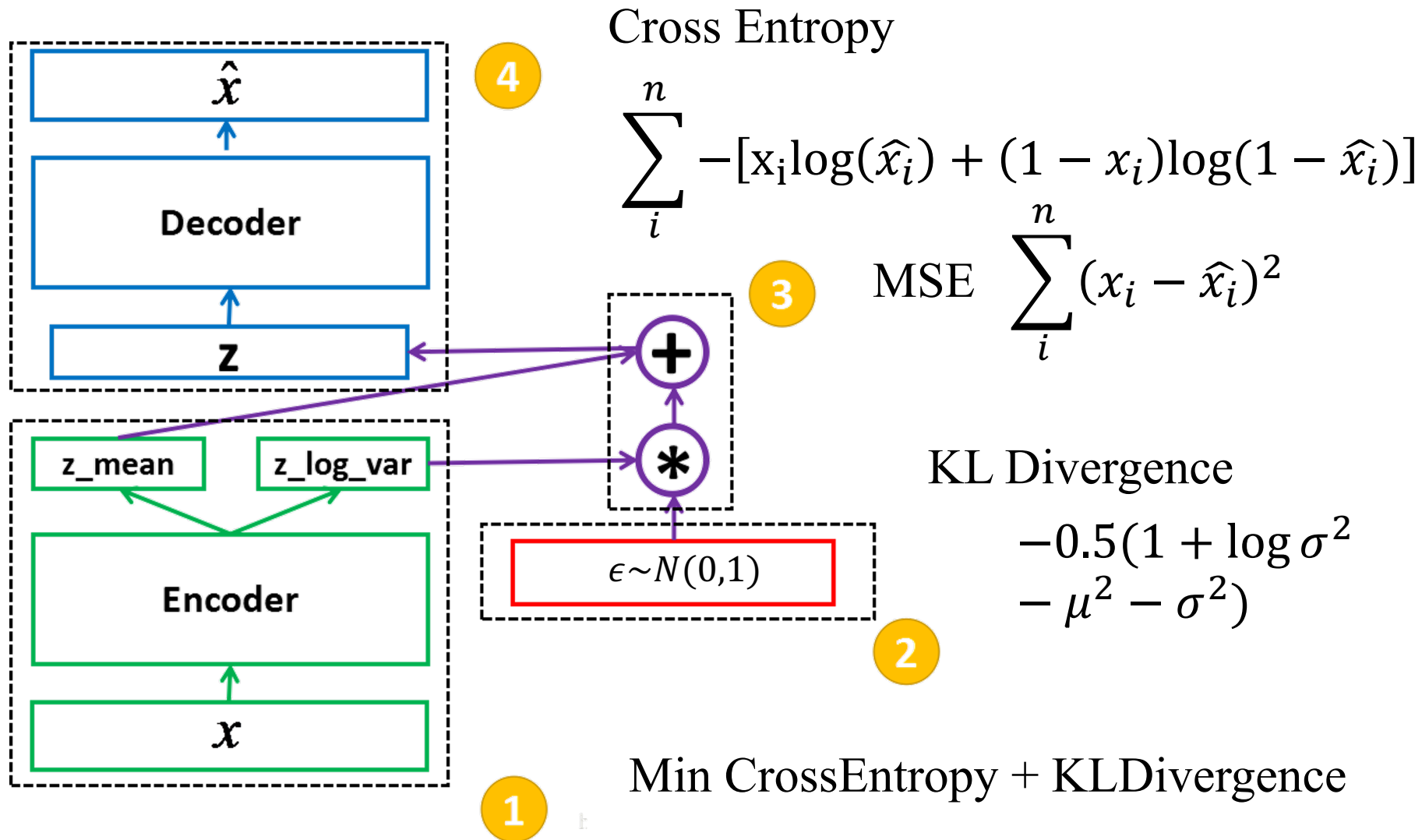


Choose prior $p(z)$ to be simple, e.g. Gaussian.
Reasonable for latent attributes, e.g. pose.

Variational Autoencoders



Variational Autoencoders

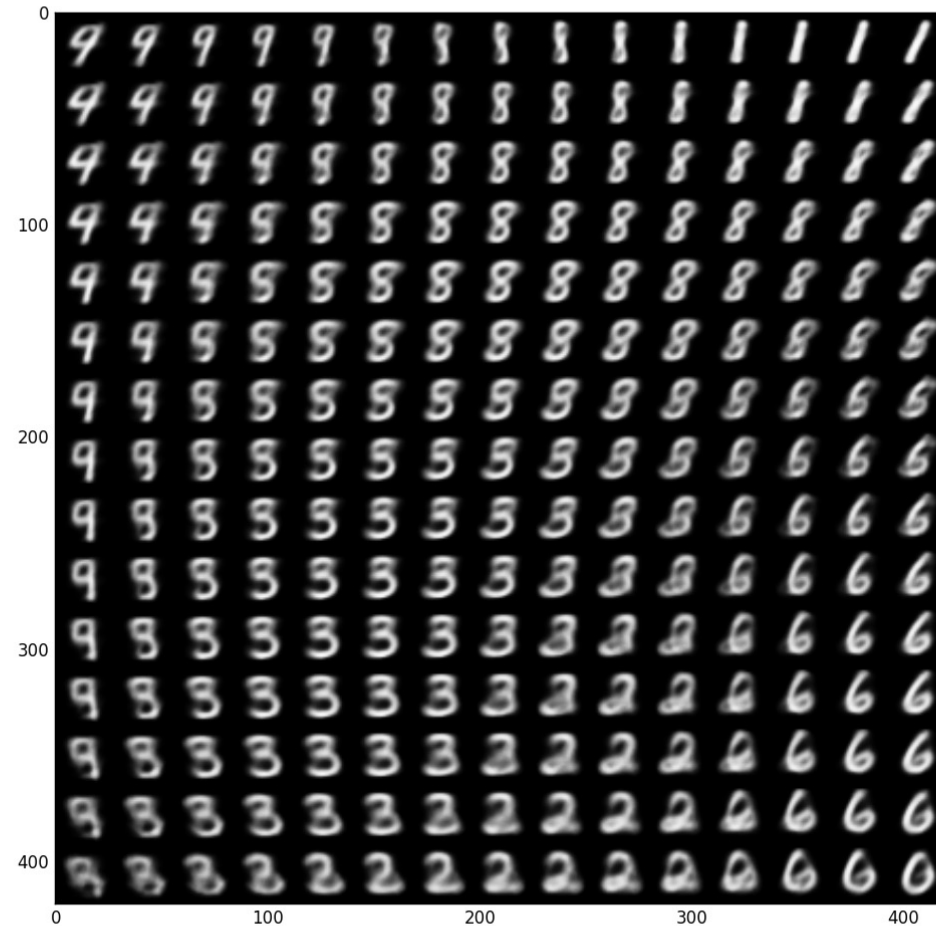
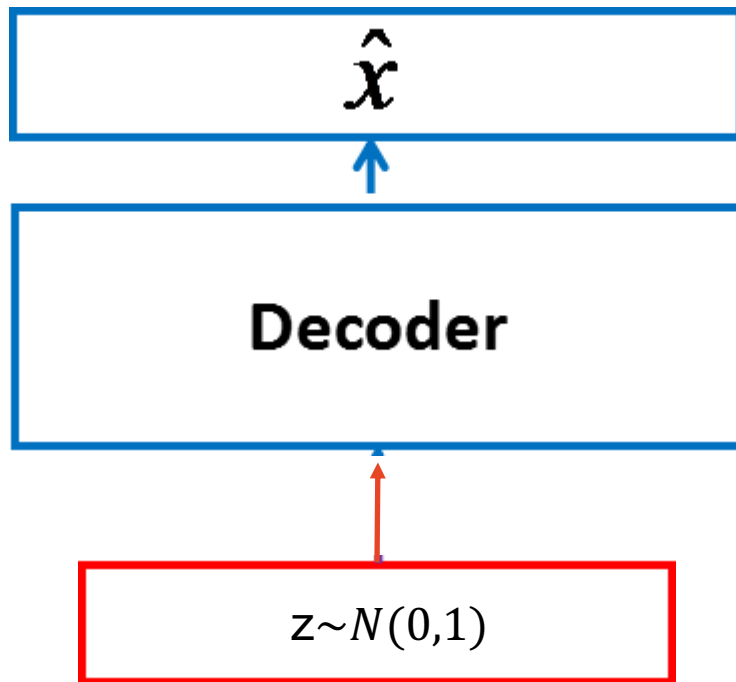


Variational Autoencoders

Generating Data:

Use decoder network.

Sample z from prior.

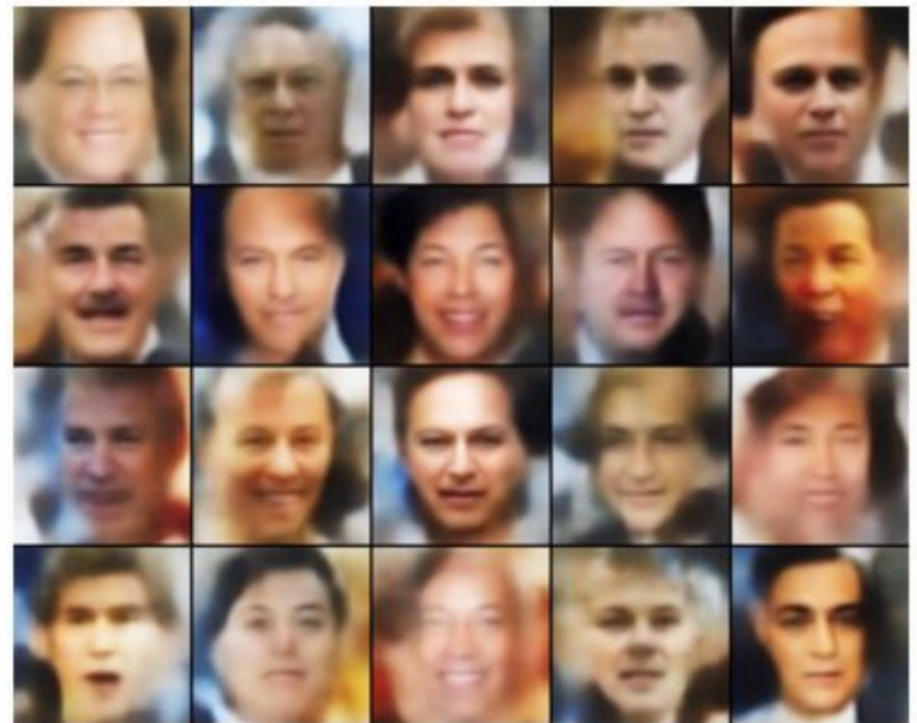


Variational Autoencoders

Generating Data:



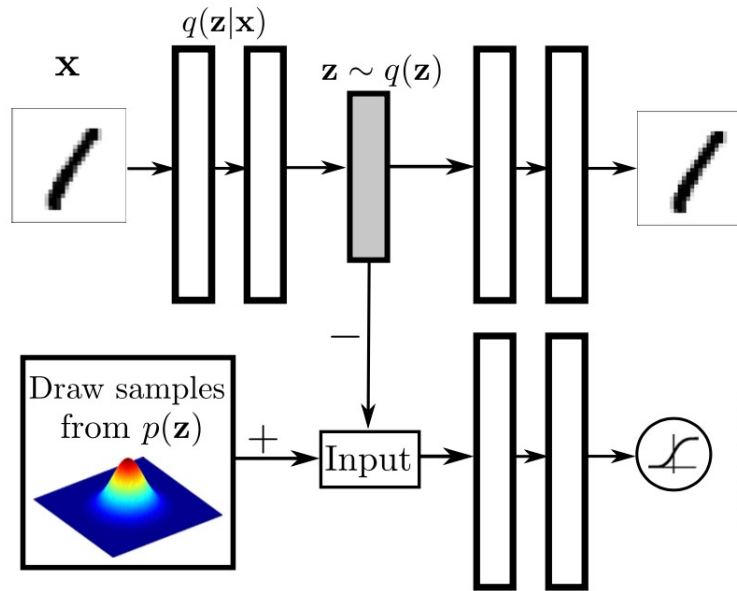
32x32 CIFAR-10



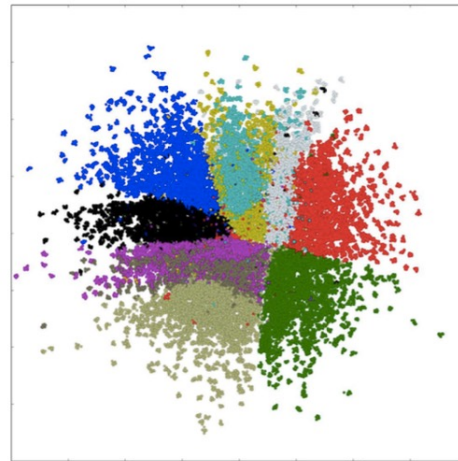
Labeled Faces in the Wild

VAE + GAN

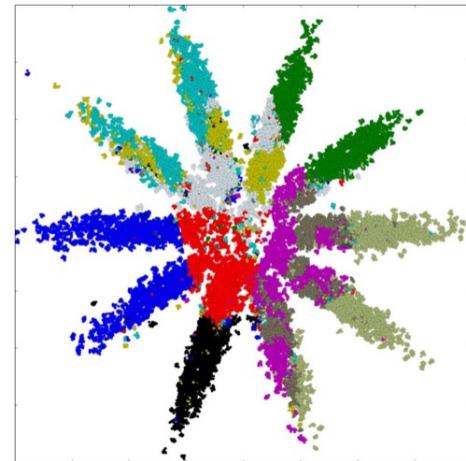
$$q(\mathbf{z}) = \int_{\mathbf{x}} q(\mathbf{z}|\mathbf{x})p_{\text{data}}(\mathbf{x})d\mathbf{x}$$



Code Space
of MNIST:



Gaussian Prior



Mixture of Gaussians

Makhzani et al., 2015

Diffusion Models

Image Synthesis



[Dhariwal and Nichol., “Diffusion Models Beat GANs on Image Synthesis”, NeurIPS 2021](#)

Super-Resolution

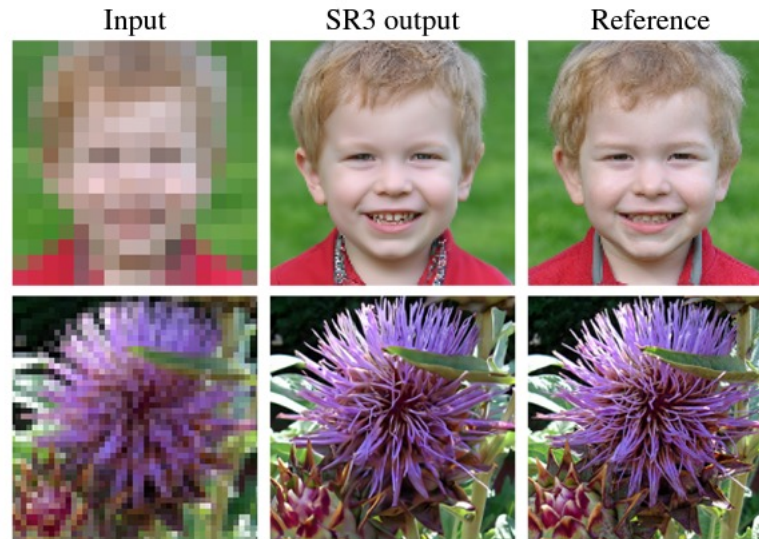


Figure 1: Two representative SR3 outputs: (top) $8\times$ face super-resolution at $16\times 16 \rightarrow 128\times 128$ pixels (bottom) $4\times$ natural image super-resolution at $64\times 64 \rightarrow 256\times 256$ pixels.

[Saharia et al., “Image Super-Resolution via Iterative Refinement”, IEEE TPAMI 2023](#)

Image-to-Image Translation

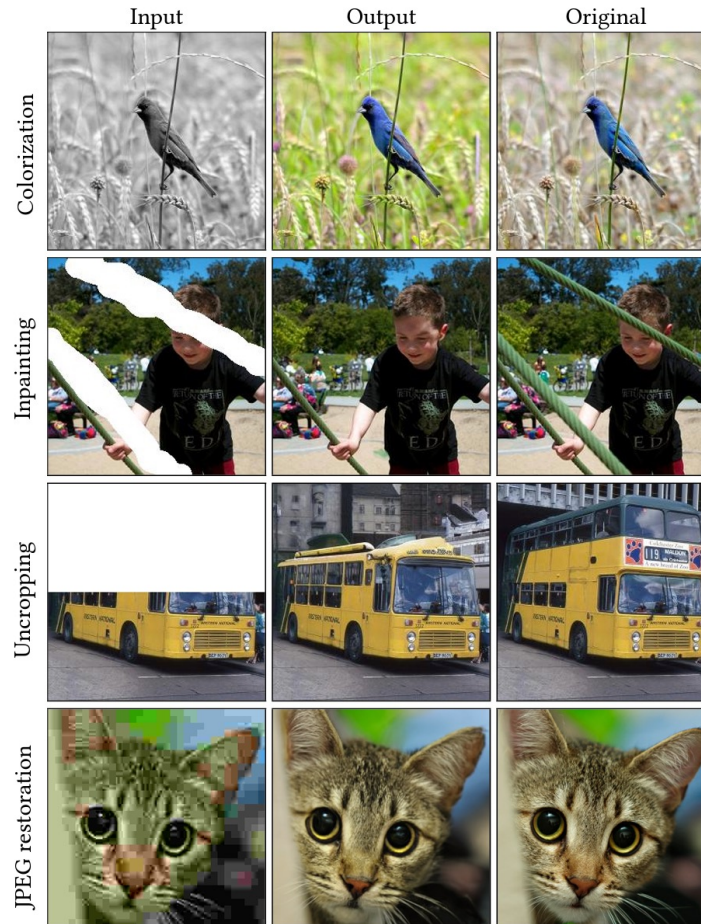


Figure 1: Image-to-image diffusion models are able to generate high-fidelity output across tasks without task-specific customization or auxiliary loss.

[Saharia et al., Palette: Image-to-Image Diffusion Models, NeurIPS 2021](#)

Text-to-Image Generation (DALL·E 2)



a photo of a cat → an anime drawing of a super saiyan cat, artstation



a photo of a victorian house → a photo of a modern house



a photo of an adult lion → a photo of lion cub



a photo of a landscape in winter → a photo of a landscape in fall

[Ramesh et al., “Hierarchical Text-Conditional Image Generation with CLIP Latents”, arXiv 2022](#)

Text-to-Image Generation (Imagen)



A dragon fruit wearing karate belt in the snow.



A brain riding a rocketship heading towards the moon.

[Saharia et al., "Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding", arXiv 2022](#)

Text-to-Animation Generation (Disco Diffusion)

Journey Across the Kingdom



Source: https://twitter.com/Somnai_dreams/status/1477525429971939332

Text-to-Video Generation



Samples from a text-conditioned video diffusion model, conditioned on the string *fireworks*.

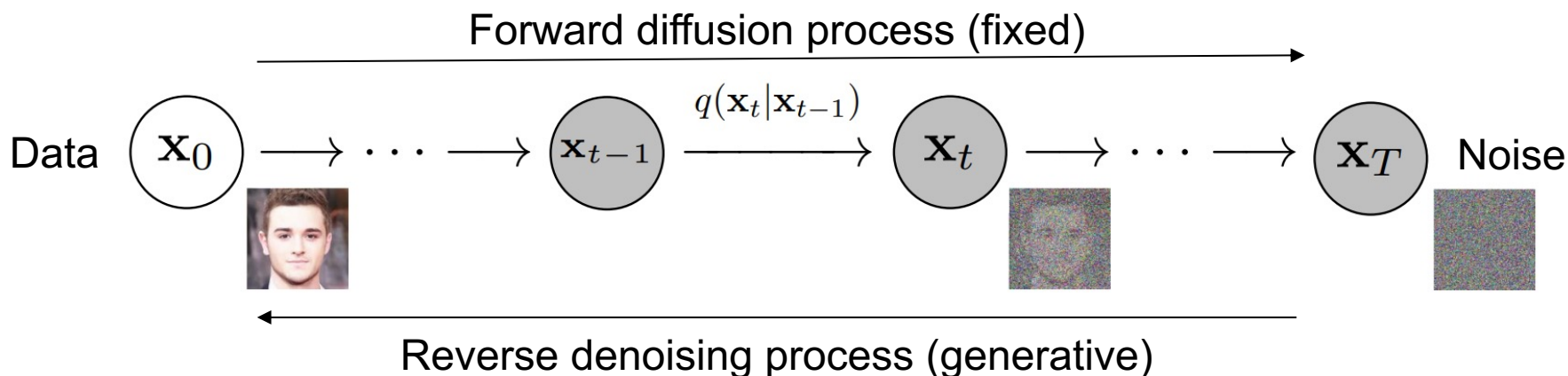
[Ho et al., "Video Diffusion Models", CVPR 2022](#)

Denoising Diffusion Probabilistic Models (DDPMs)

Overview

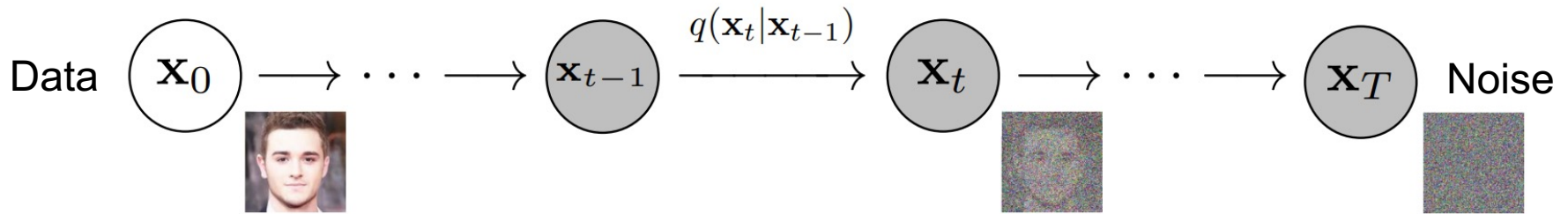
Denoising Diffusion Probabilistic Models (DDPMs) consist of 2 processes:

- **Forward process:** gradually add noise to input to destroy structure in data distribution.
- **Reverse process:** learn a denoiser (i.e., a neural network) to restore structure in data distribution.



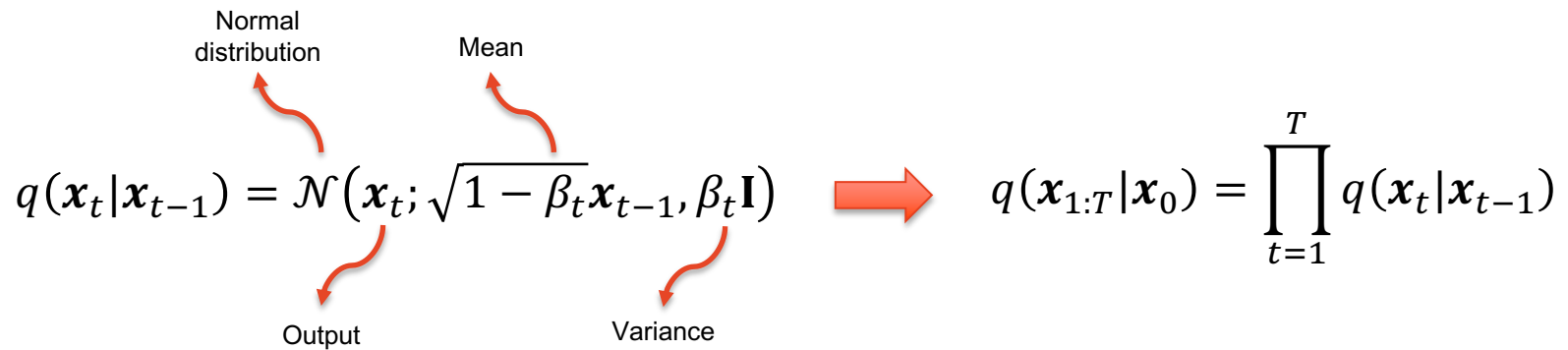
[Ho et al., “Denoising Diffusion Probabilistic Models”, NeurIPS 2020](#)

Forward Process



$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}) \quad \longrightarrow \quad q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})$$

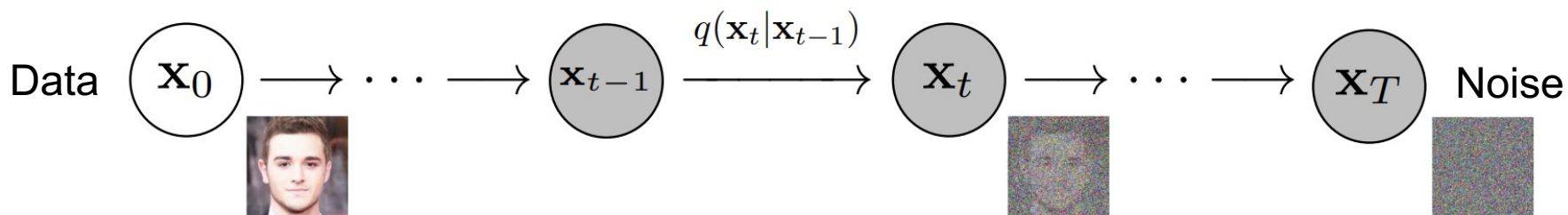
Normal distribution Mean Output Variance

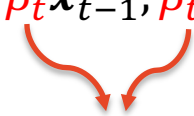


The diagram shows the mathematical formulation of the forward process. On the left, the transition probability $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ is defined as a Normal distribution $\mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$. Red arrows point from the labels 'Normal distribution', 'Mean', 'Output', and 'Variance' to the corresponding parts of the equation. A large red arrow points to the right, where the joint probability $q(\mathbf{x}_{1:T}|\mathbf{x}_0)$ is given as the product of the individual transition probabilities $\prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})$.

[Ho et al., "Denoising Diffusion Probabilistic Models", NeurIPS 2020](#)

Forward Process



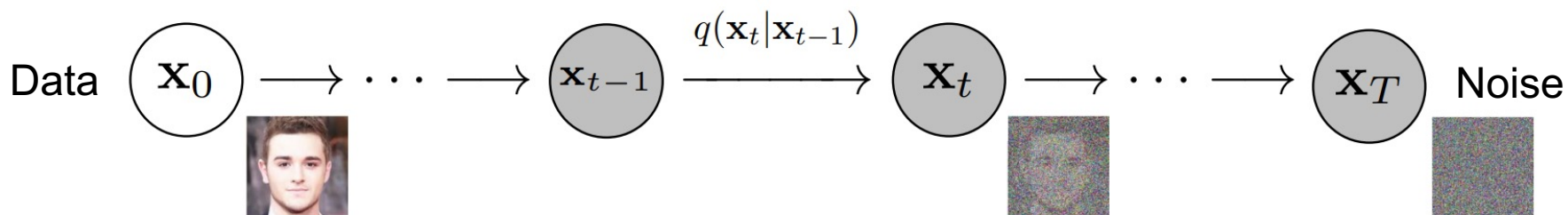
$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}) \quad \longrightarrow \quad q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})$$


$\beta_t \in (0,1)$ from $t = 0$ to $t = T$ is called the **noise schedule** and is normally a small number to ensure the data is being scaled and the variance doesn't explode.

For example, Ho et al (NeurIPS 2020) suggest β_t increases linearly from $\beta_0 = 0.0001$ to $\beta_{1000} \in 0.02$.

[Ho et al., “Denoising Diffusion Probabilistic Models”, NeurIPS 2020](#)

Forward Process



Define $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$



Can sample $\mathbf{x}_t$ at any arbitrary time step t in a closed form. See next slide for proof.

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}) \quad \text{This is called Diffusion Kernel.}$$

[Ho et al., “Denoising Diffusion Probabilistic Models”, NeurIPS 2020](#)

Proof Sketch

Define $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$

Forward Process $q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$

Using reparameterization trick:

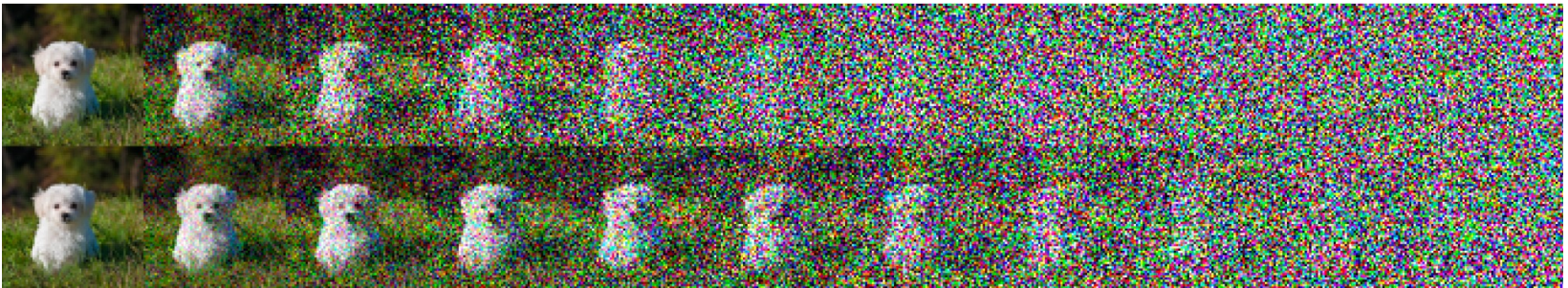
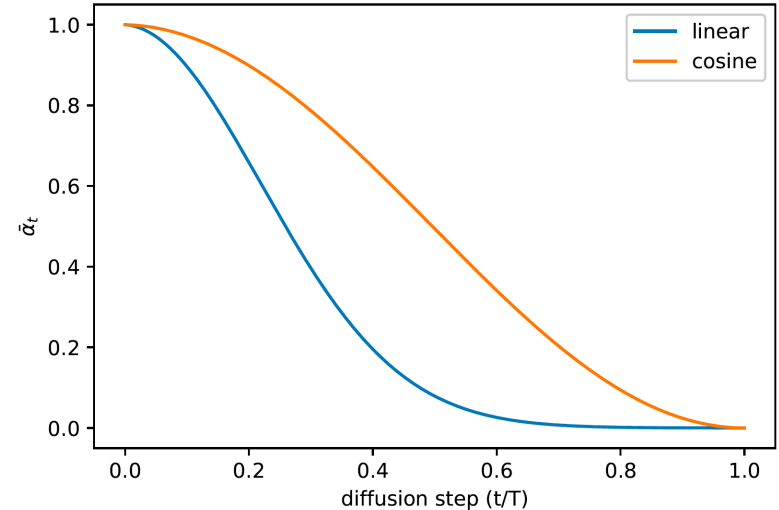
$$\begin{aligned} \mathbf{x}_t &= \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \boldsymbol{\epsilon}_{t-1} \\ &= \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-1} \\ &= \sqrt{\alpha_t} (\sqrt{\alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_{t-1}} \boldsymbol{\epsilon}_{t-2}) + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-1} \\ &= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{\alpha_t (1 - \alpha_{t-1})} \boldsymbol{\epsilon}_{t-2} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-1} \\ &= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{\alpha_t (1 - \alpha_{t-1}) + 1 - \alpha_t} \bar{\boldsymbol{\epsilon}}_{t-2} \\ &= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \bar{\boldsymbol{\epsilon}}_{t-2} \\ &= \dots \\ &= \sqrt{\alpha_t \alpha_{t-1} \dots \alpha_1} \mathbf{x}_0 + \sqrt{1 - \alpha_t \alpha_{t-1} \dots \alpha_1} \boldsymbol{\epsilon} \\ &= \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon} \end{aligned}$$

So, $q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$

Noise Schedule Improvements

- The linear schedule from Ho et al. (2020) falls towards zero much faster, destroying information more quickly than necessary.
- Dhariwal and Nichol (2021) propose a cosine noise schedule that adds noise more slowly.

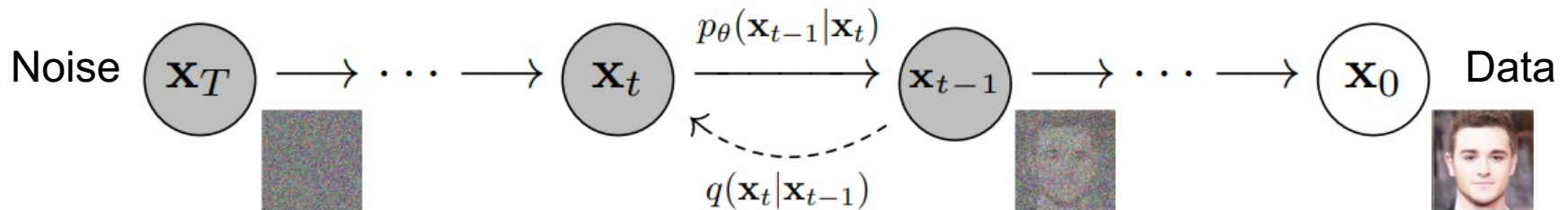
$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos\left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2}\right)^2$$



Latent samples from linear (top) and cosine (bottom) schedules respectively at linearly spaced values of t from 0 to T .

[Dhariwal and Nichol., “Improved Denoising Diffusion Probabilistic Models”, ICML 2021](#)

Reverse Process



- Ideally, we can do image generation by (1) Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$. (2) Then, iteratively sample $\mathbf{x}_{t-1} \sim q(\mathbf{x}_{t-1}|\mathbf{x}_t)$.
- If β_t is small in each forward diffusion step, $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ would be Gaussian.
- We learn a denoiser model $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ to approximate $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$.

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

- Note that the reverse conditional probability **is tractable when also conditioning on $\mathbf{x}_0$** (see next slide for proof):

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I})$$

$\tilde{\boldsymbol{\mu}}(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_t \right)$

$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \cdot \beta_t$

Ho et al., “Denoising Diffusion Probabilistic Models”, NeurIPS 2020

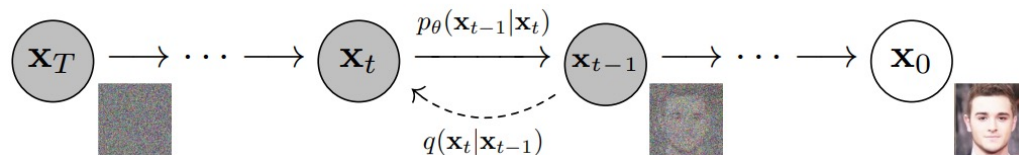
Proof Sketch

Recall $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I})$

Using Bayes rule:

$$\begin{aligned}
 & \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{q(\mathbf{x}_t, \mathbf{x}_0)} = q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) \frac{q(\mathbf{x}_{t-1}, \mathbf{x}_0)}{q(\mathbf{x}_t, \mathbf{x}_0)} = q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)} \\
 & \propto \exp \left(-\frac{1}{2} \left[\frac{(\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1})^2}{\beta_t} + \frac{(\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0)^2}{1 - \bar{\alpha}_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0)^2}{1 - \bar{\alpha}_t} \right] \right) \\
 & = \exp \left(-\frac{1}{2} \left[\frac{\mathbf{x}_t^2 - 2\sqrt{\alpha_t} \mathbf{x}_t \mathbf{x}_{t-1} + \alpha_t \mathbf{x}_{t-1}^2}{\beta_t} + \frac{\mathbf{x}_{t-1}^2 - 2\sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0 \mathbf{x}_{t-1} + \bar{\alpha}_{t-1} \mathbf{x}_0^2}{1 - \bar{\alpha}_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0)^2}{1 - \bar{\alpha}_t} \right] \right) \\
 & = \exp \left(-\frac{1}{2} \left[\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} \right) \mathbf{x}_{t-1}^2 - \left(\frac{2\sqrt{\alpha_t}}{\beta_t} + \frac{2\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} \mathbf{x}_0 \right) \mathbf{x}_{t-1} + C(\mathbf{x}_t, \mathbf{x}_0) \right] \right) \\
 & = \exp \left(-\frac{1}{2} \left[\frac{\mathbf{x}_{t-1} - \left(\frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 \right)}{\frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \cdot \beta_t} \right]^2 \right) \begin{matrix} \xrightarrow{\text{red}} \tilde{\boldsymbol{\mu}}(\mathbf{x}_t, \mathbf{x}_0) \\ \xrightarrow{\text{blue}} \tilde{\beta}_t \end{matrix}
 \end{aligned}$$

Proof Sketch (cont.)



Recall the Diffusion Kernel: $q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$

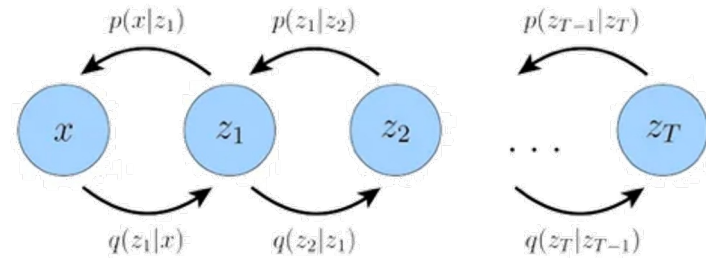
We have: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\varepsilon}$

Thus, $\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\varepsilon}_t)$

We can then plug it in the equation of $\tilde{\mu}(\mathbf{x}_t, \mathbf{x}_0)$:

$$\begin{aligned}\tilde{\mu}(\mathbf{x}_t, \mathbf{x}_0) &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\mathbf{x}_0 \\ &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\left(\frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\varepsilon}_t)\right) \\ &= \frac{1}{\sqrt{\bar{\alpha}_t}}\left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}}\boldsymbol{\varepsilon}_t\right)\end{aligned}$$

Learning Objective



Like for training variational autoencoders, we can use the variational lower bound to optimize the negative log-likelihood:

$$-\mathbb{E}_{q(\mathbf{x}_0)}[\log p_{\theta}(\mathbf{x}_0)] \leq \mathbb{E}_{q(\mathbf{x}_{0:T})} \left[\log \frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] =: L$$

Sohl-Dickstein et al. (2015) and Ho et al. (2020) show that

$$L = \mathbb{E}_q \left[\underbrace{D_{KL}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} - \underbrace{\log p_{\theta}(\mathbf{x}_0|\mathbf{x}_1)}_{L_0} \right]$$

Since both $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ and $p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$ are normal distributions, the KL divergence has a simple form:

$$\begin{aligned} L_{t-1} &= D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)) \\ &= \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(0, \mathbf{I})} \left[\frac{1}{2 \|\boldsymbol{\Sigma}_{\theta}(\mathbf{x}_t, t)\|^2} \|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t)\|^2 \right] + C \end{aligned}$$

[Ho et al., “Denoising Diffusion Probabilistic Models”, NeurIPS 2020](#)

Learning Objective

Recall we have:

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\varepsilon}_t \right)$$

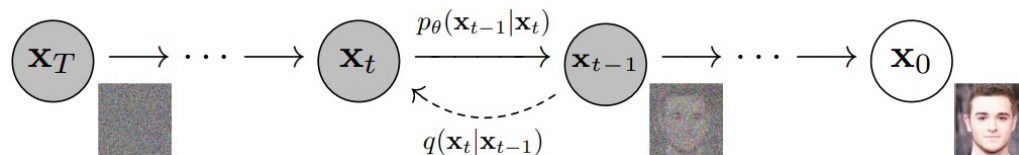
We can reparametrize to make the model predict noise $\boldsymbol{\varepsilon}_t$ to denoise from $\mathbf{x}_t$ and obtain $\mathbf{x}_{t-1}$ (instead of directly predicting sample $\mathbf{x}_{t-1}$) at each diffusion step t :

$$\tilde{\mu}_\theta(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t) \right)$$

$$\begin{aligned} L_{t-1} &= D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)) \\ &= \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \boldsymbol{\varepsilon} \sim \mathcal{N}(0, \mathbf{I})} \left[\frac{1}{2 \|\boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)\|^2} \|\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)\|^2 \right] + C \\ &= \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\varepsilon}} \left[\frac{1}{2 \|\boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)\|^2} \left\| \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\varepsilon}_t \right) - \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t) \right) \right\|^2 \right] + C \\ &= \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\varepsilon}} \left[\frac{(1 - \alpha_t)^2}{2 \alpha_t (1 - \bar{\alpha}_t) \|\boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)\|^2} \|\boldsymbol{\varepsilon}_t - \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)\|^2 \right] + C \\ &= \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\varepsilon}} \left[\frac{(1 - \alpha_t)^2}{2 \alpha_t (1 - \bar{\alpha}_t) \|\boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)\|^2} \|\boldsymbol{\varepsilon}_t - \boldsymbol{\varepsilon}_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}_t, t)\|^2 \right] + C \end{aligned}$$

Ho et al., “Denoising Diffusion Probabilistic Models”, NeurIPS 2020

Algorithm



Empirically, Ho et al. (2020) found diffusion models can be trained with a simplified objective that ignores the weighting term $\frac{(1-\alpha_t)^2}{2\alpha_t(1-\bar{\alpha}_t)\|\Sigma_\theta(\mathbf{x}_t, t)\|^2}$. They also set $\Sigma_\theta(\mathbf{x}_t, t) = \sigma_t^2 \mathbf{I}$, where σ_t is not learned but set to β_t .

$$L_t^{simple} = \mathbb{E}_{t \sim [1, T], \mathbf{x}_0, \epsilon} \left[\left\| \epsilon_t - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon_t, t) \right\|^2 \right] + \mathcal{C}$$

Algorithm 1 Training

- 1: **repeat**
- 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
- 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
- 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 5: Take gradient descent step on $\nabla_\theta \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$
- 6: **until** converged

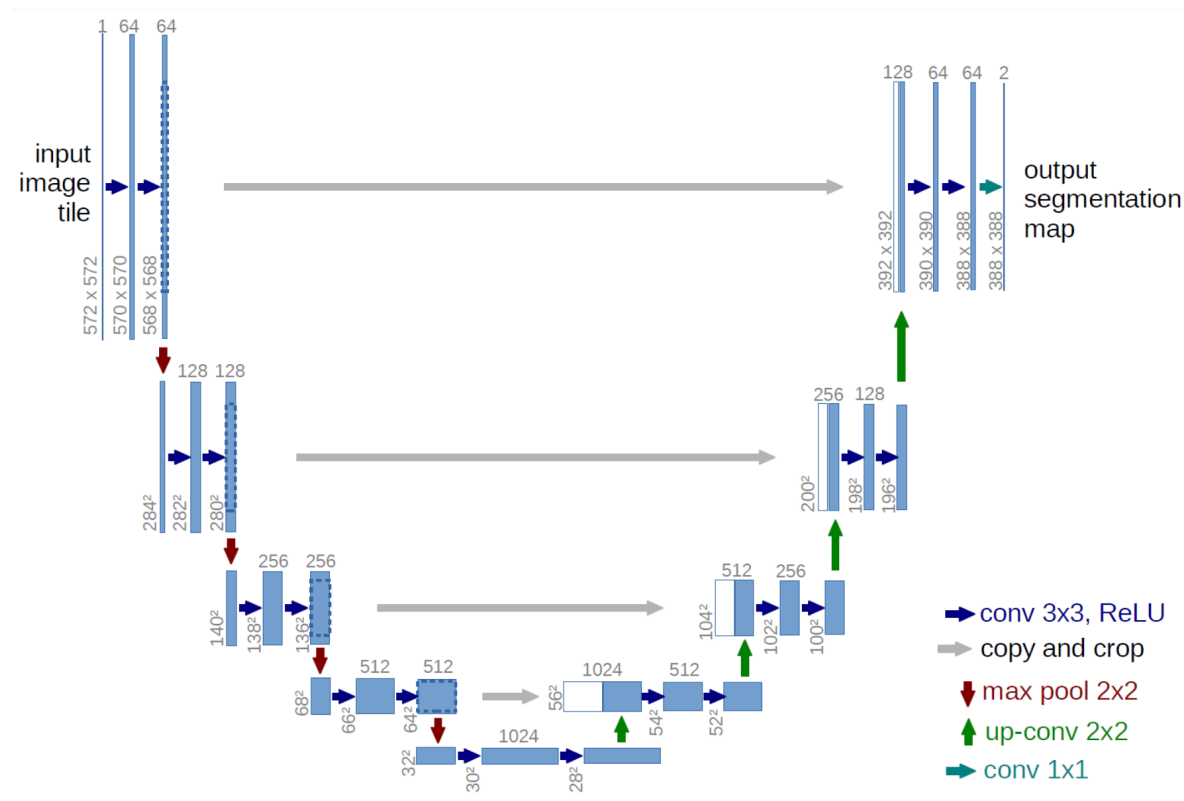
Algorithm 2 Sampling

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 2: **for** $t = T, \dots, 1$ **do**
- 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
- 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
- 5: **end for**
- 6: **return** $\mathbf{x}_0$

Ho et al., “Denoising Diffusion Probabilistic Models”, NeurIPS 2020

Network Architectures

Diffusion models often use U-Net architectures with ResNet blocks and self-attention layers to represent $\epsilon_{\theta}(\mathbf{x}_t, t)$. Time representations are normally sinusoidal positional embeddings or random Fourier features and are fed to the residual blocks using either simple spatial addition or using adaptive group normalization layers.



Ronneberger et al., "U-Net: Convolutional Networks for Biomedical Image Segmentation", MICCAI 2015

Classifier-Free Guidance (CFG)

Conditional diffusion models generate samples conditioned on text or labels:

$$\epsilon_{\theta}(x_t, t, c)$$

However, generated results may not strongly follow the condition.

Key Idea

Train one diffusion model with **random condition dropout**:

Conditional prediction

$$\epsilon_{cond} = \epsilon_{\theta}(x_t, t, c)$$

Unconditional prediction

$$\epsilon_{uncond} = \epsilon_{\theta}(x_t, t, \emptyset) :$$

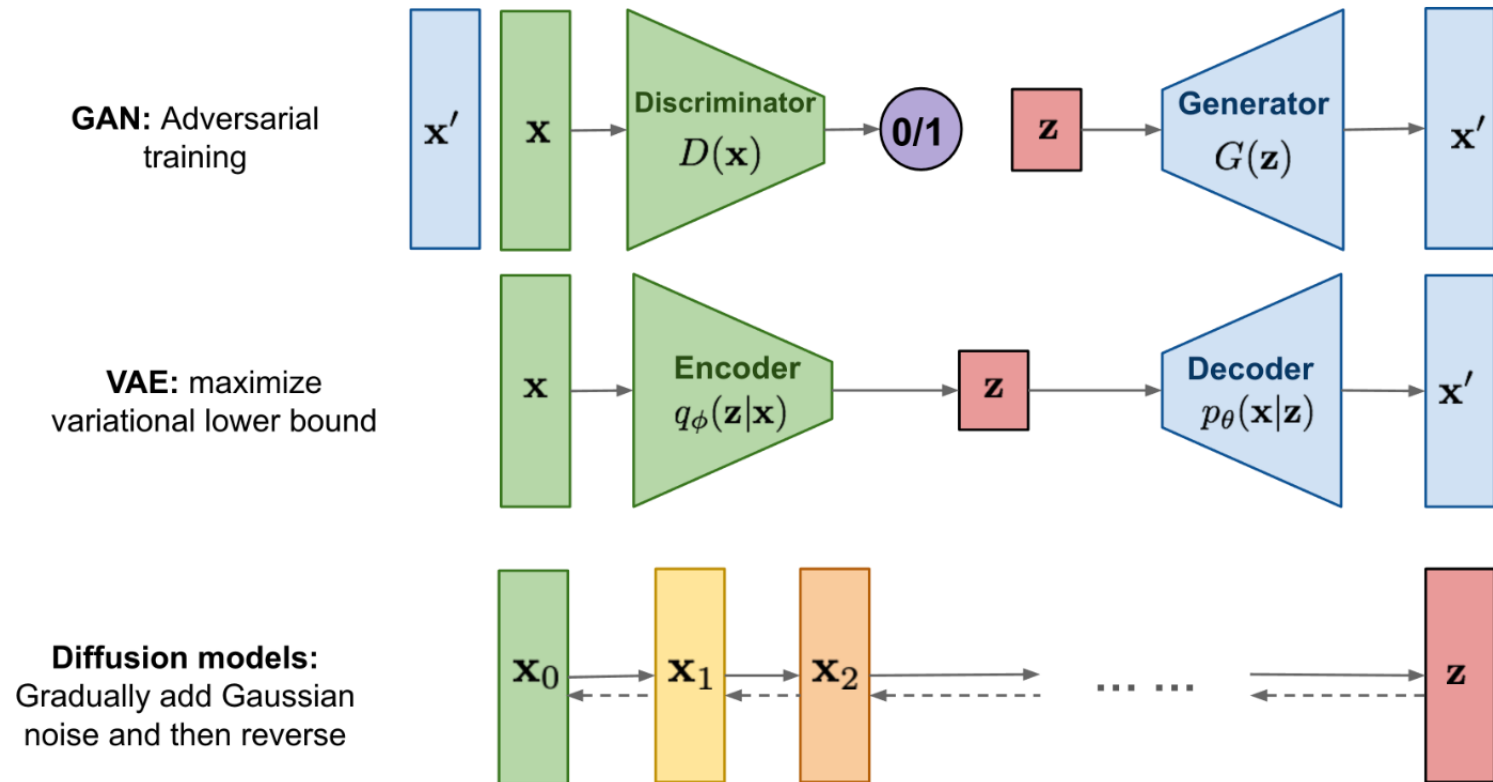
During sampling, combine them:

$$\hat{\epsilon} = \epsilon_{uncond} + \omega(\epsilon_{cond} - \epsilon_{uncond})$$

where:

- $\omega = 1$: standard conditional sampling
- $\omega > 1$: stronger guidance
- very large ω : oversaturation / artifacts

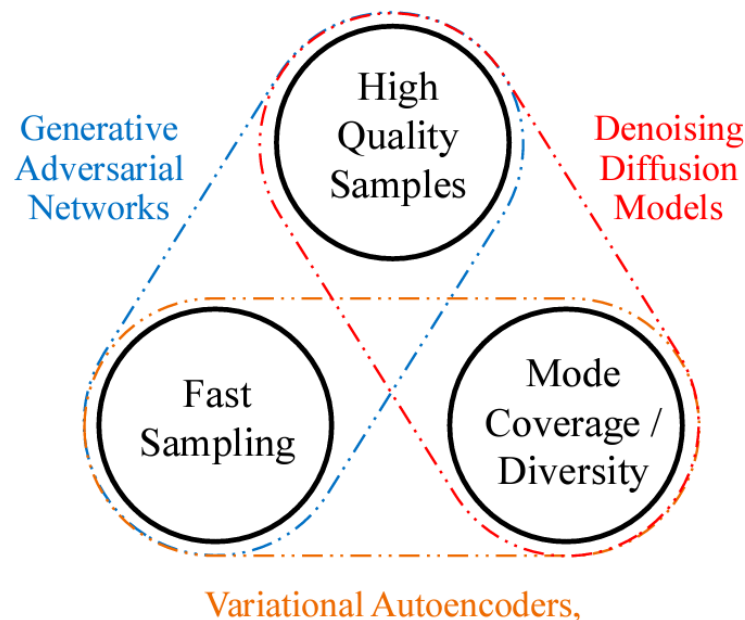
Comparing Generative Models



Weng, "What are Diffusion Models?", Lil'Log, 2021

The generative learning trilemma

- Dhariwal and Nichol (2021) has shown that Diffusion Models beats GANs in terms of sample quality and diversity. However, the inference time for Diffusion Models is much longer than GANs.
- Accelerating Diffusion Models sampling is an active area of research and has already made much progresses.

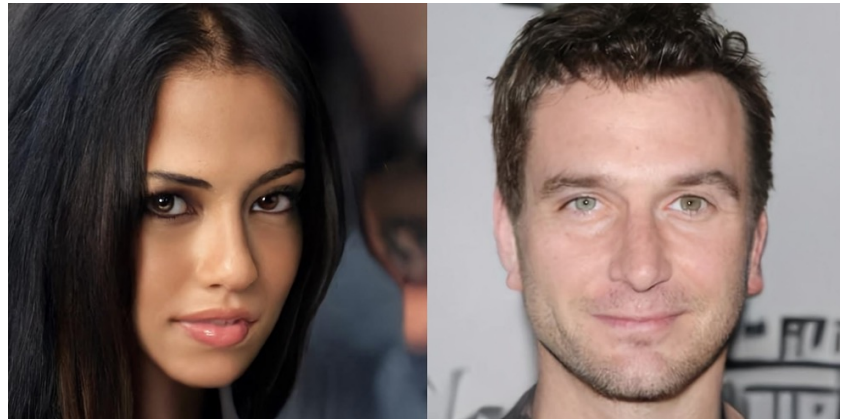


[Xiao et al., “Tackling the Generative Learning Trilemma with Denoising Diffusion GANs”, ICLR 2022](#)

Continuous View of Diffusion Models

Score-based models: impressive results

- GAN-like quality and better, while having the advantages of explicit probabilistic models
 - Explicit likelihood computation
 - Representation learning
- State-of-the-art results in generation, audio synthesis, shape generation, etc



[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

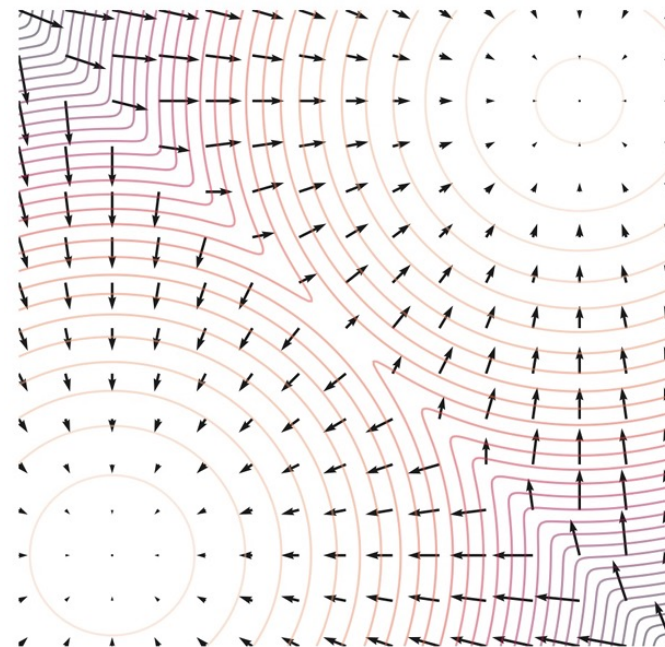
Score function

- The score function is the gradient of the log-probability of a distribution w.r.t to the input

$$\nabla_x \log p(x)$$

- A model $s_\theta(x)$, which models the score function explicitly, is a score-based model

$$s_\theta(x) \approx \nabla_x \log p(x)$$

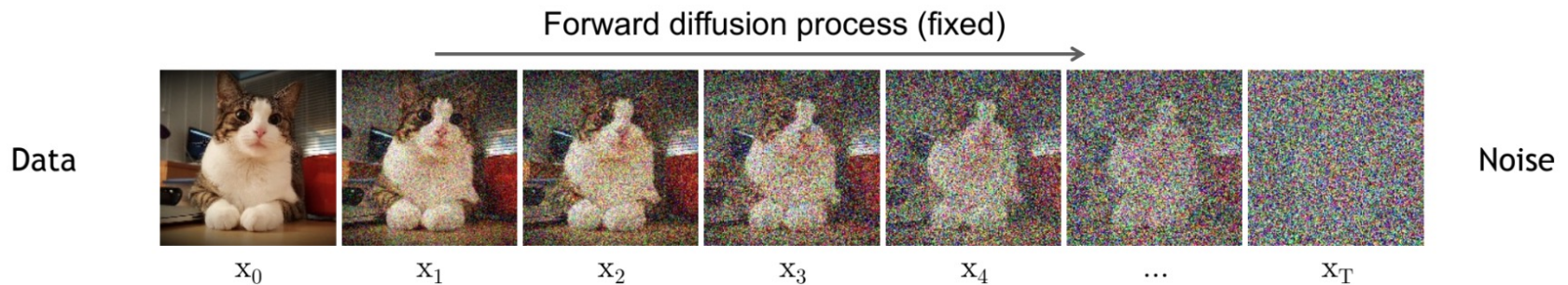


The score function of a mixture of two Gaussians

[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

Forward Diffusion Process

Consider the forward diffusion process again:

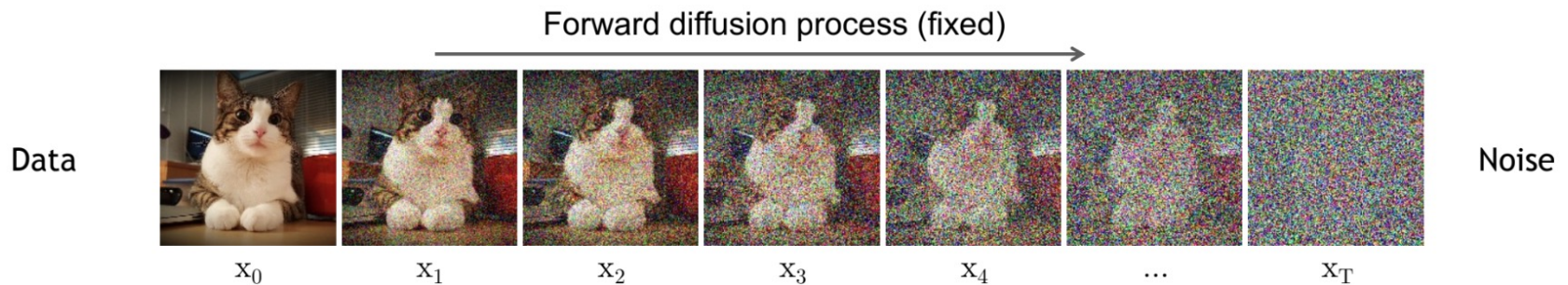


$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

Forward Diffusion Process

Consider the forward diffusion process again:



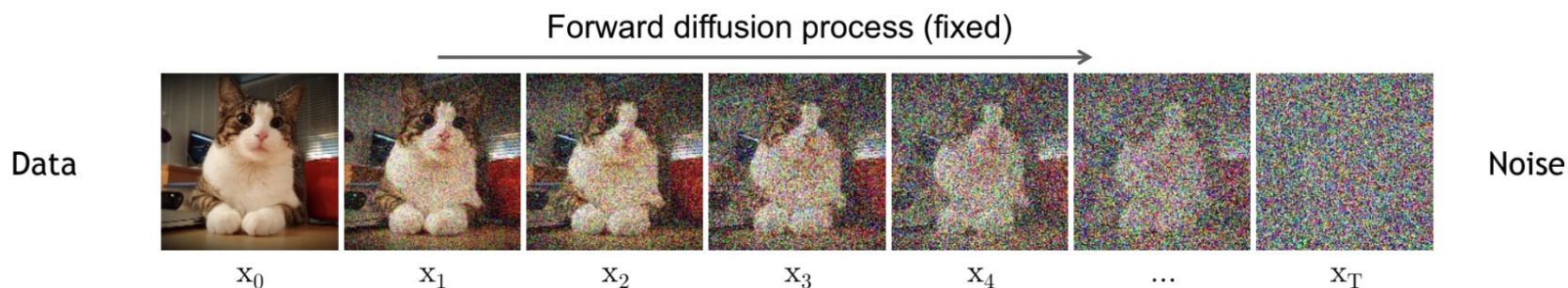
$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

→
$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \mathcal{N}(\mathbf{0}, \mathbf{I})$$

[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

Forward Diffusion Process

Consider the forward diffusion process again:



$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \mathcal{N}(\mathbf{0}, \mathbf{I})$$

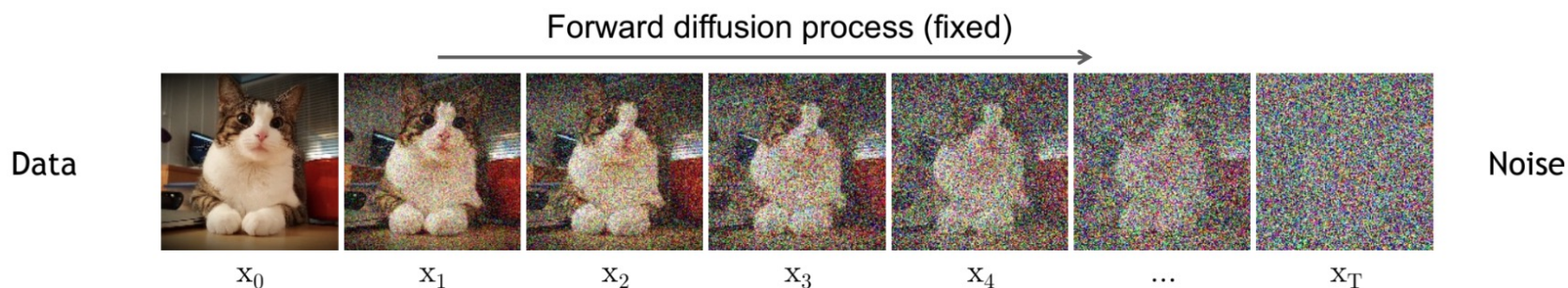


$$= \sqrt{1 - \beta(t)\Delta t} \mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t} \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (\beta_t := \beta(t)\Delta t)$$

[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

Forward Diffusion Process

Consider the forward diffusion process again:



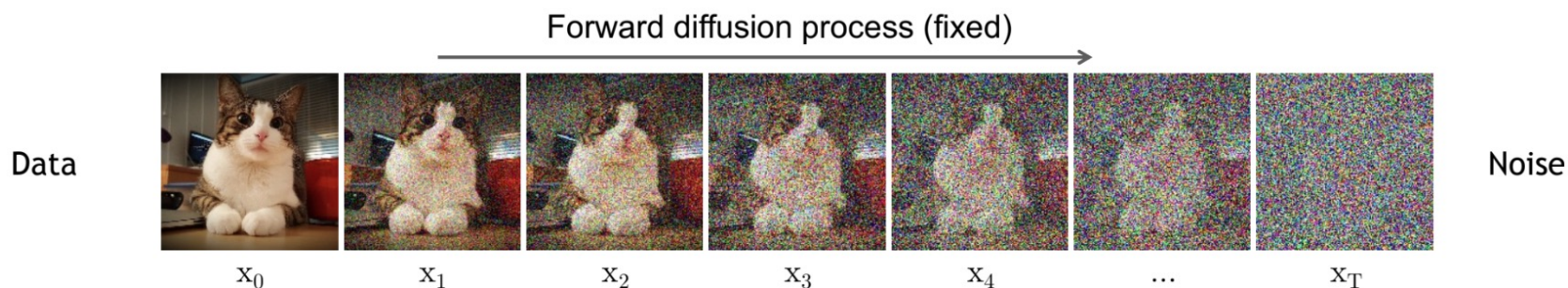
$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

$$\begin{aligned} \mathbf{x}_t &= \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \mathcal{N}(\mathbf{0}, \mathbf{I}) \\ &= \sqrt{1 - \beta(t)\Delta t} \mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t} \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (\beta_t := \beta(t)\Delta t) \\ &\approx \mathbf{x}_{t-1} - \frac{\beta(t)\Delta t}{2} \mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t} \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (\text{Taylor expansion}) \end{aligned}$$

[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

Forward Diffusion Process

Consider the forward diffusion process again:



$$\mathbf{x}_t \approx \mathbf{x}_{t-1} - \frac{\beta(t)\Delta t}{2} \mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t} \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$\Delta t \rightarrow 0$ ↓

$$d\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t dt + \sqrt{\beta(t)} d\omega_t \sim \mathcal{N}(\mathbf{0}, \Delta t)$$

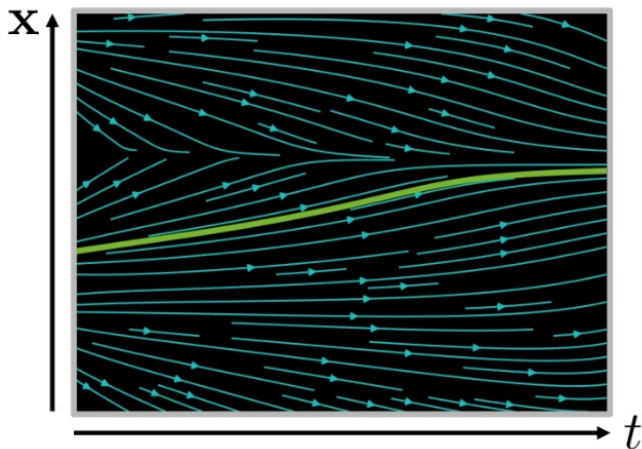
Stochastic Differential Equation (SDE)
describing the diffusion in infinitesimal limit

[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

Crash Course in Differential Equations

Ordinary Differential Equation (ODE):

$$\frac{dx}{dt} = f(x, t) \quad \text{or} \quad dx = f(x, t)dt$$

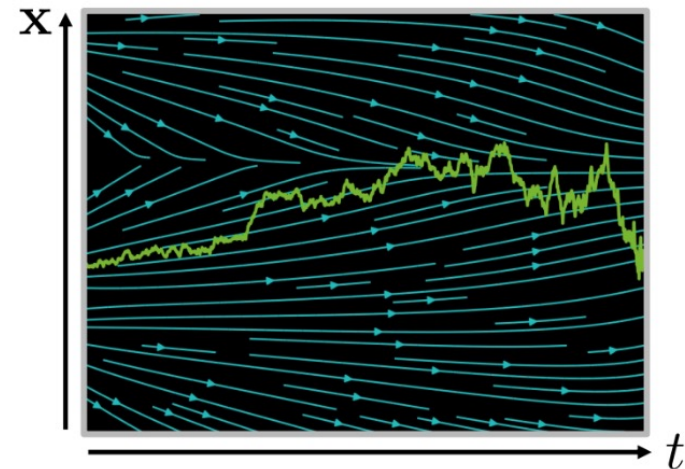


Analytical Solution:
$$\mathbf{x}(t) = \mathbf{x}(0) + \int_0^t \mathbf{f}(\mathbf{x}, \tau) d\tau$$

Iterative Numerical Solution:
$$\mathbf{x}(t + \Delta t) \approx \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t)\Delta t$$

Stochastic Differential Equation (SDE):

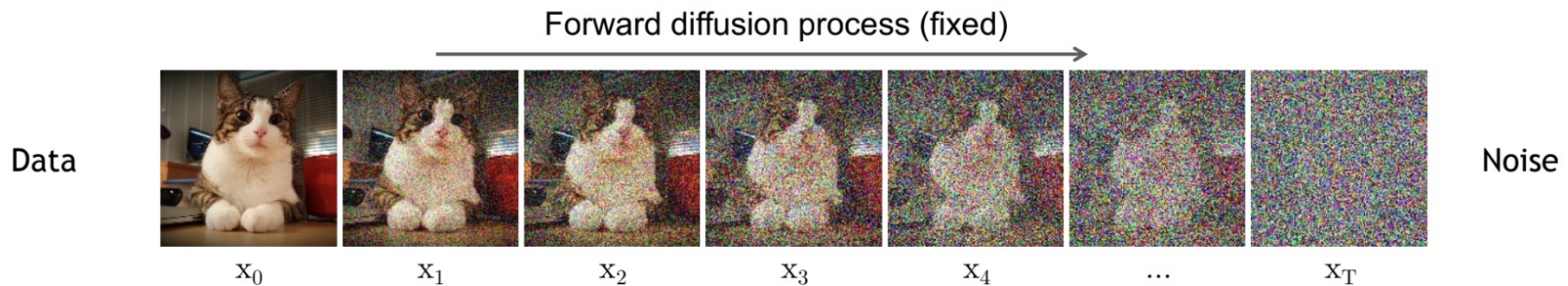
$$\frac{dx}{dt} = \underbrace{f(x, t)}_{\text{drift coefficient}} + \underbrace{\sigma(x, t)\omega_t}_{\text{diffusion coefficient}}$$



$$\mathbf{x}(t + \Delta t) \approx \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t)\Delta t + \sigma(\mathbf{x}(t), t)\sqrt{\Delta t}\mathcal{N}(\mathbf{0}, \mathbf{I})$$

Forward Diffusion Process as Stochastic Differential Equation

Consider the limit of **many small steps**:



$$\mathbf{x}_t \approx \mathbf{x}_{t-1} - \frac{\beta(t)\Delta t}{2} \mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t} \mathcal{N}(\mathbf{0}, \mathbf{I})$$

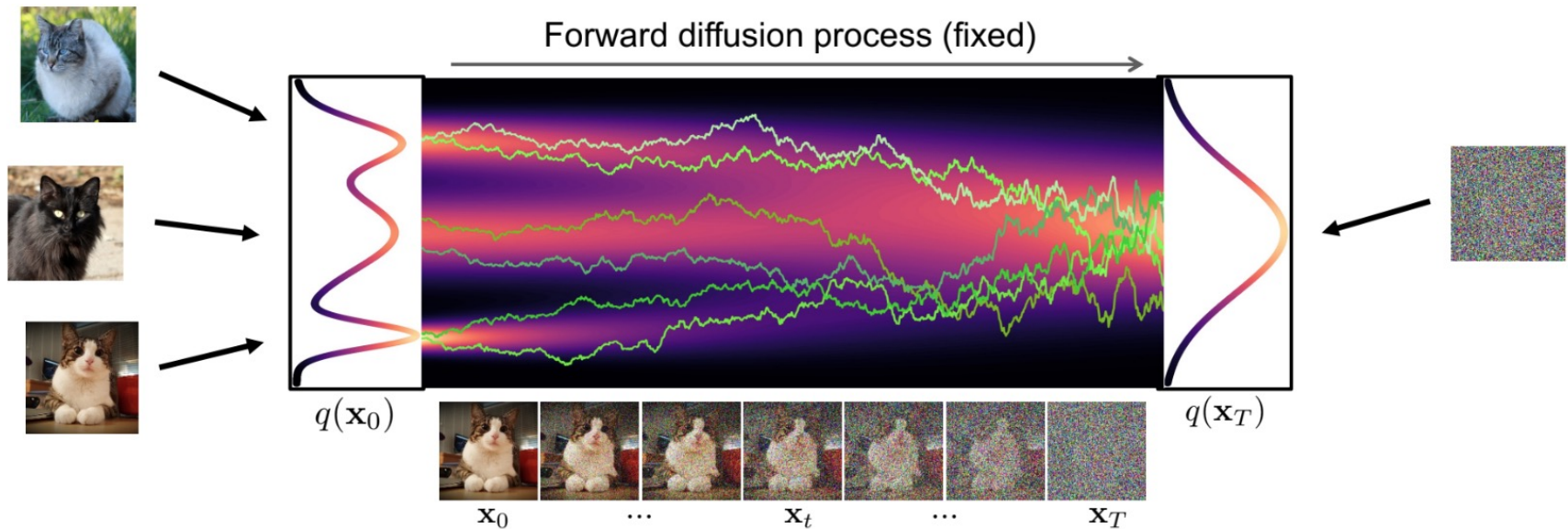


$$d\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t dt + \sqrt{\beta(t)}d\boldsymbol{\omega}_t$$

Stochastic Differential Equation
(SDE)
describing the diffusion in infinitesimal limit

[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

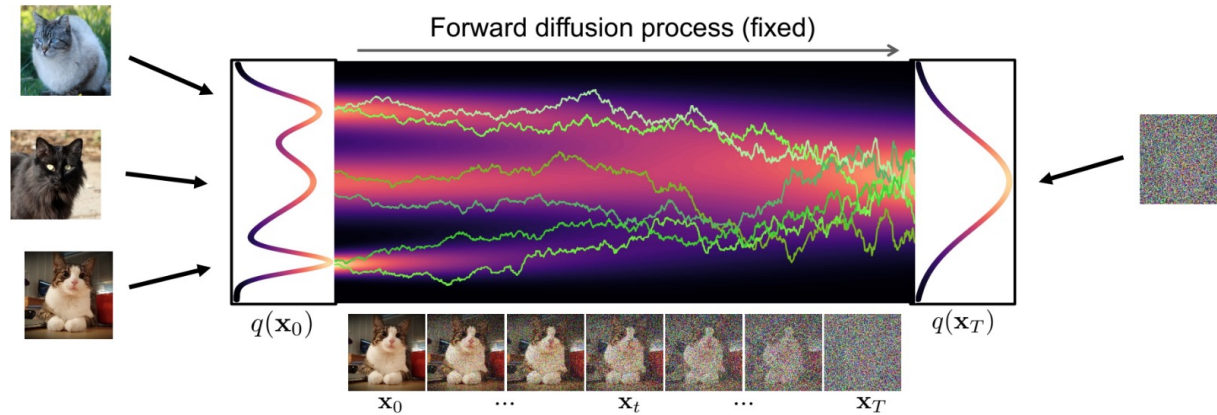
Forward Diffusion Process as Stochastic Differential Equation



Forward Diffusion SDE:
$$d\mathbf{x}_t = \underbrace{-\frac{1}{2}\beta(t)\mathbf{x}_t dt}_{\text{drift term (pulls towards mode)}} + \underbrace{\sqrt{\beta(t)}d\omega_t}_{\text{diffusion term (injects noise)}}$$

[Song et al., "Score-Based Generative Modeling through Stochastic Differential Equations", ICLR 2021](#)

The Generative Reverse Stochastic Differential Equation

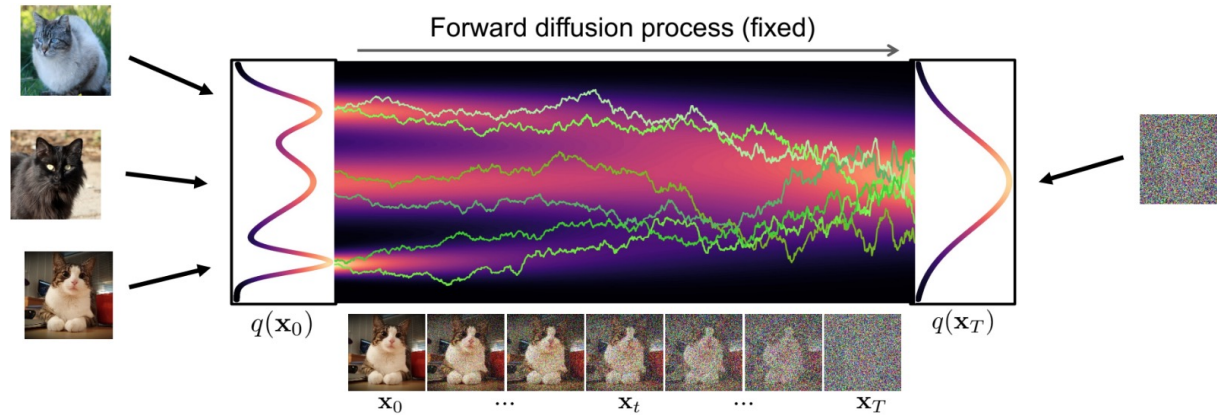


Forward Diffusion SDE:
$$d\mathbf{x}_t = \underbrace{-\frac{1}{2}\beta(t)\mathbf{x}_t dt}_{\text{drift term (pulls towards mode)}} + \underbrace{\sqrt{\beta(t)}d\omega_t}_{\text{diffusion term (injects noise)}}$$

**But what about the reverse direction,
Necessary for generation?**

[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

The Generative Reverse Stochastic Differential Equation



Forward Diffusion SDE:

$$d\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t dt + \sqrt{\beta(t)}d\boldsymbol{\omega}_t$$

Reverse Generative Diffusion SDE:

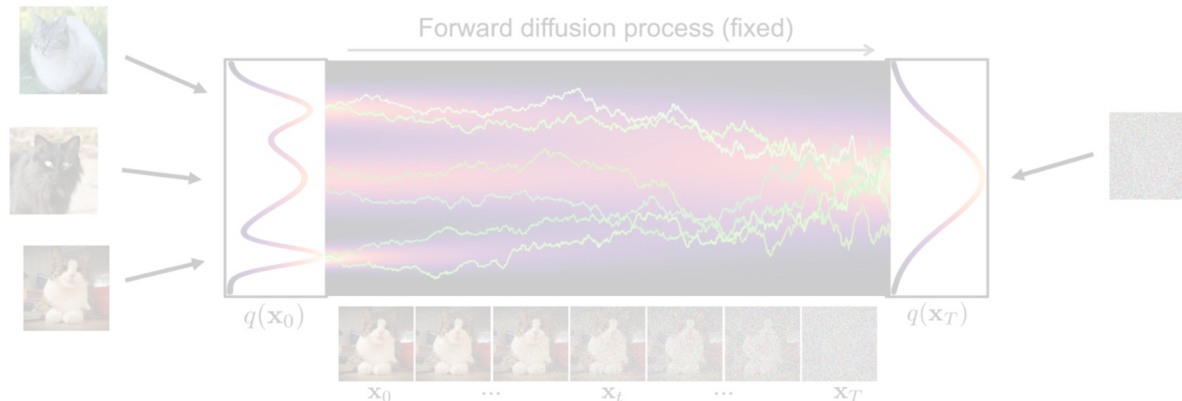
$$d\mathbf{x}_t = \underbrace{\left[-\frac{1}{2}\beta(t)\mathbf{x}_t - \underbrace{\beta(t)\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)}_{\text{score function}} \right]}_{\text{drift term}} dt + \underbrace{\sqrt{\beta(t)}d\bar{\boldsymbol{\omega}}_t}_{\text{diffusion term}}$$



Simulate reverse diffusion process: Data generation from random noise!

[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

The Generative Reverse Stochastic Differential Equation



But how to get the score function $\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)$?

Forward Diffusion SDE: $d\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t dt + \sqrt{\beta(t)}d\omega_t$

Reverse Generative
Diffusion SDE:

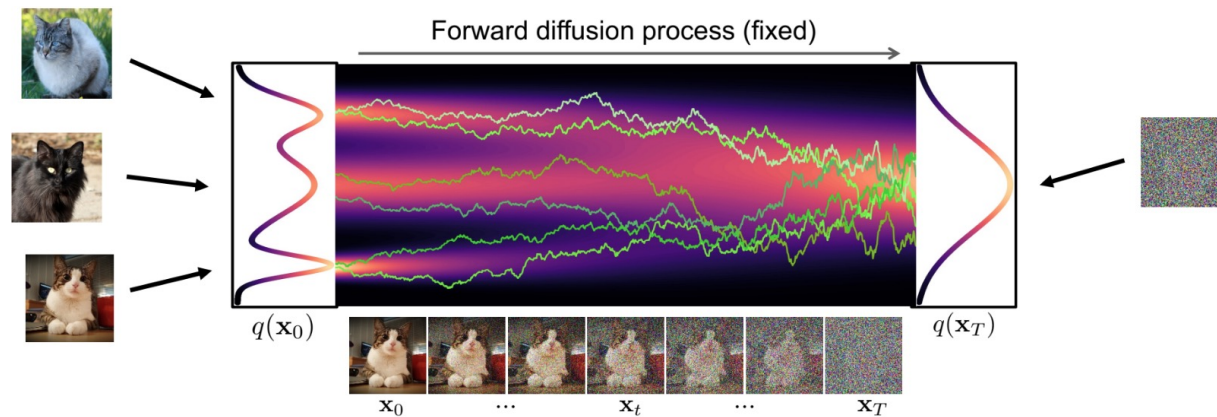
$$d\mathbf{x}_t = \underbrace{\left[-\frac{1}{2}\beta(t)\mathbf{x}_t - \underbrace{\beta(t)\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)}_{\text{score function}} \right]}_{\text{drift term}} dt + \underbrace{\sqrt{\beta(t)}d\bar{\omega}_t}_{\text{diffusion term}}$$



Simulate reverse diffusion process: Data generation from random noise!

Song et al., "Score-Based Generative Modeling through Stochastic Differential Equations", ICLR 2021

Score Matching



Naïve idea, learn model for the score function by direct regression?

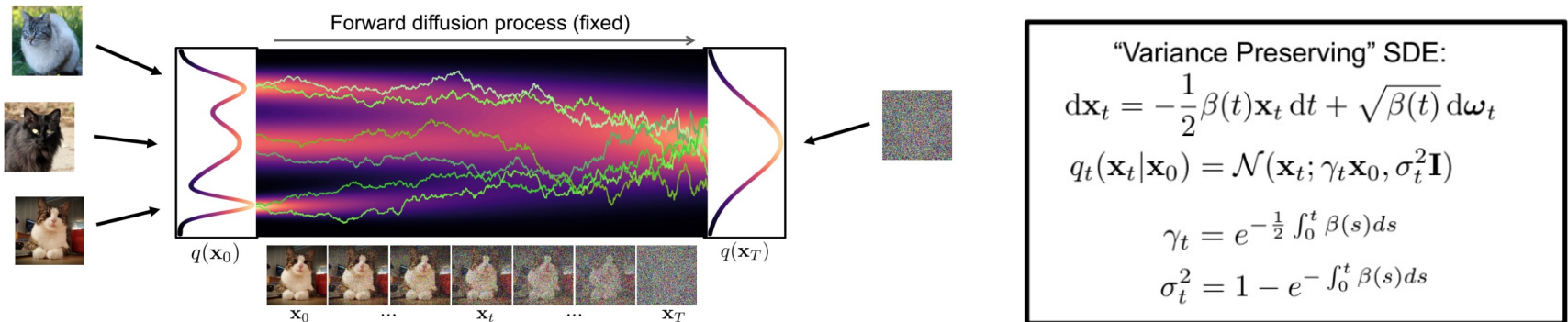
$$\min_{\theta} E_{t \sim \mathcal{U}(\mathbb{T}, T)} E_{\mathbf{x}_t \sim q_t(\mathbf{x}_t)} \|\mathbf{s}_{\theta}(\mathbf{x}_t, t) - \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)\|_2^2$$

Diffusion
Time t Diffused
data $\mathbf{x}_t$ Neural
networkScore of
diffused data
(marginal)

➡ But $\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)$ (score of the marginal diffused density $q_t(\mathbf{x}_t)$) is not tractable!

[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

Score Matching



Instead, diffuse individual data points $\mathbf{x}_0$. Diffused $q_t(\mathbf{x}_t|\mathbf{x}_0)$ is tractable!

Denoising Score Matching:

$$\min_{\theta} E_{t \sim \mathbb{U}(\mathbb{0}, T)} E_{\mathbf{x}_0 \sim q_0(\mathbf{x}_0)} E_{\mathbf{x}_t \sim q_t(\mathbf{x}_t|\mathbf{x}_0)} || \underbrace{\mathbf{s}_{\theta}(\mathbf{x}_t, t)}_{\text{Neural network}} - \underbrace{\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t|\mathbf{x}_0)}_{\text{Score of diffused data}} ||_2^2$$

Diffusion Time t
Data sample $\mathbf{x}_t$
Diffused data sample $\mathbf{x}_t$
Neural network
Score of diffused data

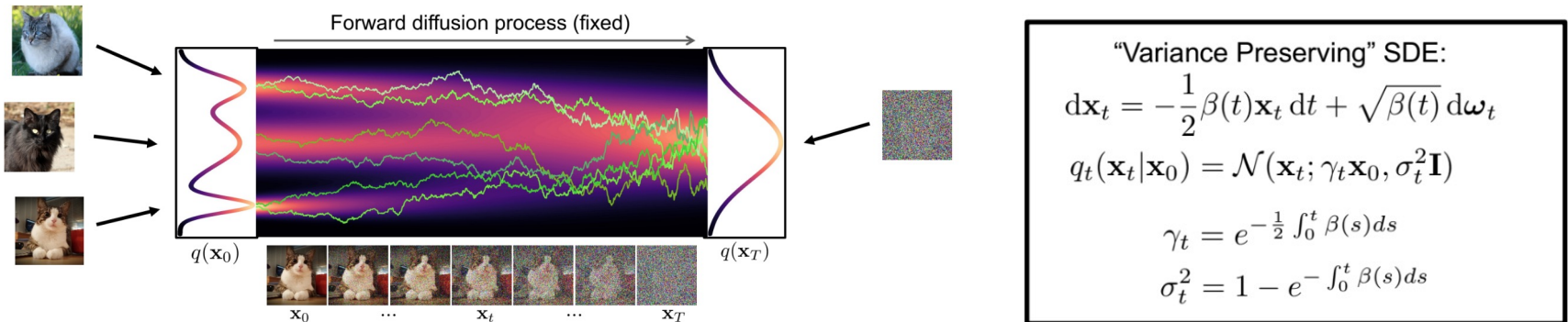


After expectations, $\mathbf{s}_{\theta}(\mathbf{x}_t, t) \approx \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)$!

[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

Score Matching

Implementation 1: Noise Prediction



Denoising Score Matching:

$$\min_{\theta} E_{t \sim \mathbb{U}(\mathbb{0}, T)} E_{x_0 \sim q_0(x_0)} E_{x_t \sim q_t(x_t|x_0)} \|s_{\theta}(x_t, t) - \nabla_{x_t} \log q_t(x_t|x_0)\|_2^2$$

Re-parametrized sampling: $x_t = \gamma_t x_0 + \sigma_t \epsilon \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

Score function: $\nabla_{x_t} \log q_t(x_t | x_0) = -\nabla_{x_t} \frac{(x_t - \gamma_t x_0)^2}{2\sigma_t^2} = -\frac{x_t - \gamma_t x_0}{\sigma_t^2} = -\frac{\gamma_t x_0 + \sigma_t \epsilon - \gamma_t x_0}{\sigma_t^2} = -\frac{\epsilon}{\sigma_t}$

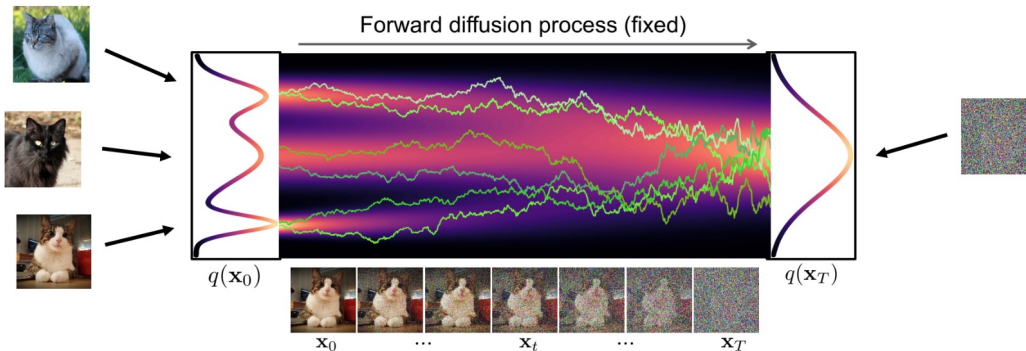
Neural network model: $s_{\theta}(x_t, t) := -\frac{\epsilon_{\theta}(x_t, t)}{\sigma_t}$

➡ $\min_{\theta} E_{t \sim \mathbb{U}(0, T)} E_{x_0 \sim q_0(x_0)} E_{\epsilon \sim \mathcal{N}(0, I)} \frac{1}{\sigma_t^2} \|\epsilon - \epsilon_{\theta}(x_t, t)\|_2^2$

Song et al., "Score-Based Generative Modeling through Stochastic Differential Equations", ICLR 2021

Score Matching

Implementation 2: Loss Weightings



“Variance Preserving” SDE:

$$d\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t dt + \sqrt{\beta(t)} d\boldsymbol{\omega}_t$$

$$q_t(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \gamma_t\mathbf{x}_0, \sigma_t^2\mathbf{I})$$

$$\gamma_t = e^{-\frac{1}{2} \int_0^t \beta(s) ds}$$

$$\sigma_t^2 = 1 - e^{-\int_0^t \beta(s) ds}$$

Denoising Score Matching objective with loss weighting $\lambda(t)$:

$$\min_{\theta} E_{t \sim \mathbb{U}(0,T)} E_{x_0 \sim q_0(x_0)} E_{\epsilon \sim \mathcal{N}(0,I)} \frac{\lambda(t)}{\sigma_t^2} \|\epsilon - \epsilon_{\theta}(x_t, t)\|_2^2$$

Different loss weightings trade off between model with good perceptual quality vs. high log-likelihood

- *Perceptual quality:* $\lambda(t) = \sigma_t^2$
- *Maximum log-likelihood:* $\lambda(t) = \beta(t)$ (negative ELBO)

[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

Key Limitations of SDE

$$dx_t = -\frac{1}{2}\beta(t)[x_t + 2\nabla_{x_t} \log q_t(x_t)]dt + \sqrt{\beta(t)}d\bar{\omega}_t$$

- **Stochastic trajectories**

- Non-deterministic generation
- Hard to control / invert

- **No tractable likelihood**

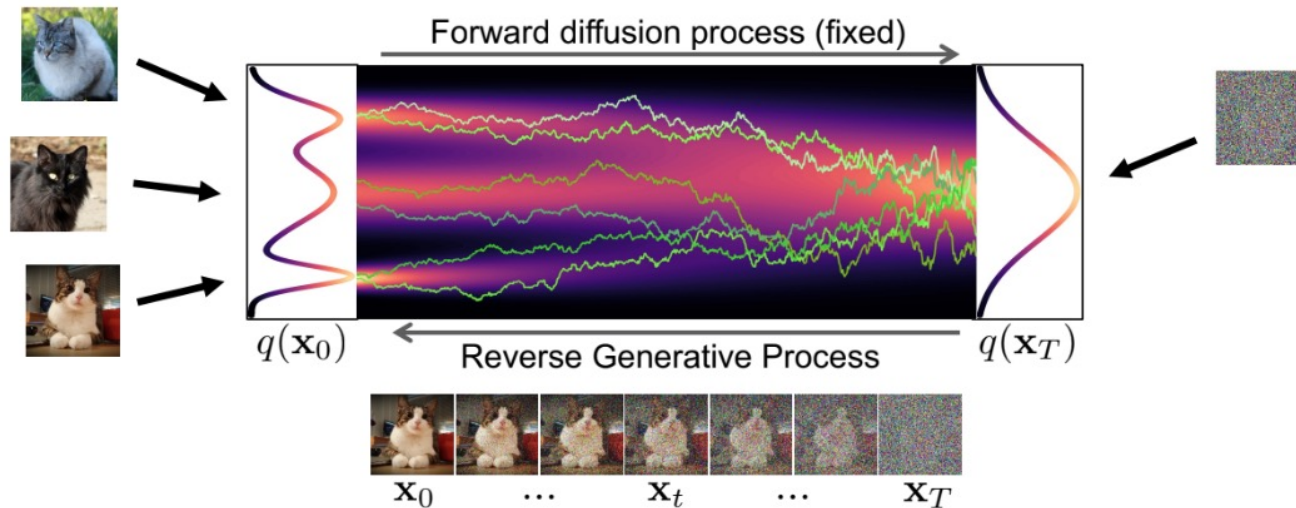
- Difficult for evaluation and downstream tasks

- **Inefficient sampling**

- Requires many small steps (e.g., 1000+)

Can we construct a **deterministic system** that produces the *same distribution evolution* as the SDE?

Probability Flow ODE



- Consider reverse generative diffusion SDE:

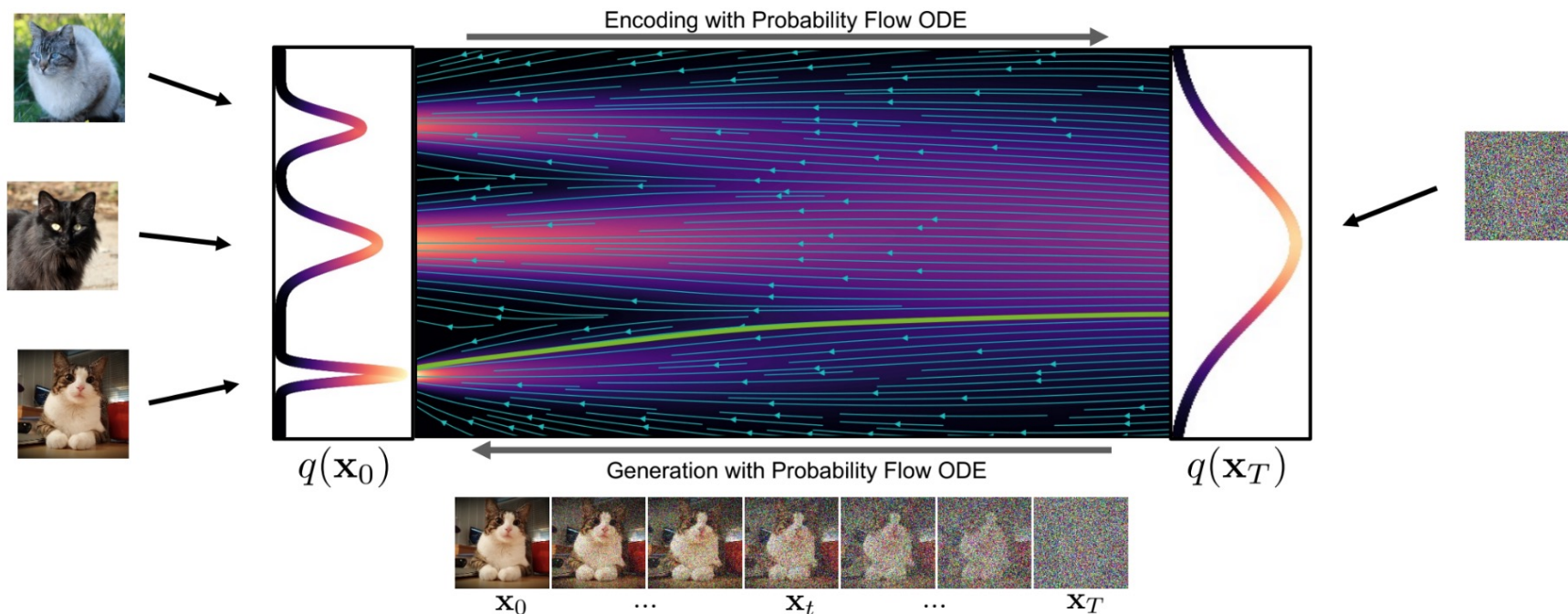
$$dx_t = -\frac{1}{2}\beta(t)[x_t + 2\nabla_{x_t} \log q_t(x_t)]dt + \sqrt{\beta(t)}d\bar{\omega}_t$$

- In distribution equivalent to “**Probability Flow ODE**”:

$$dx_t = -\frac{1}{2}\beta(t)[x_t + \nabla_{x_t} \log q_t(x_t)]dt$$

[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

Probability Flow ODE



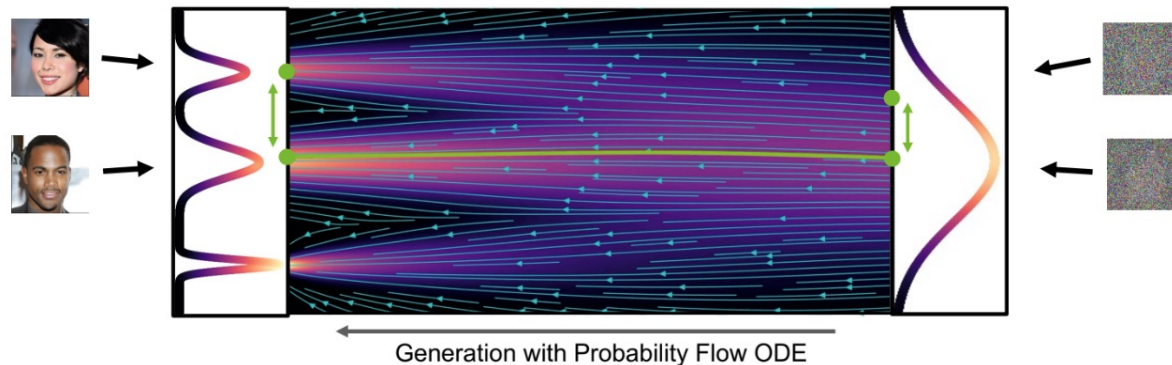
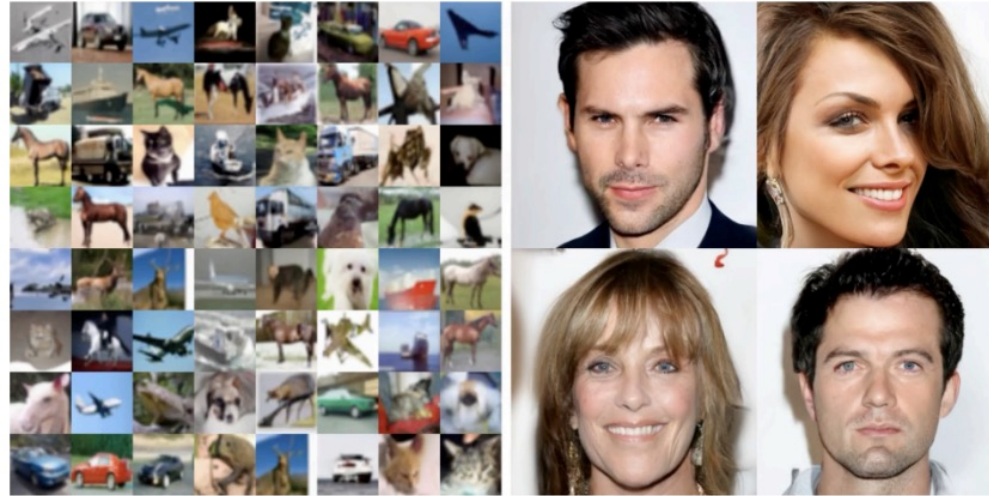
Enables use of **advanced ODE solvers**

$$dx_t = -\frac{1}{2}\beta(t)[x_t + s_\theta(x_t, t)]dt$$

Deterministic encoding and generation
(semantic image interpolation, etc.)

[Song et al., "Score-Based Generative Modeling through Stochastic Differential Equations", ICLR 2021](#)

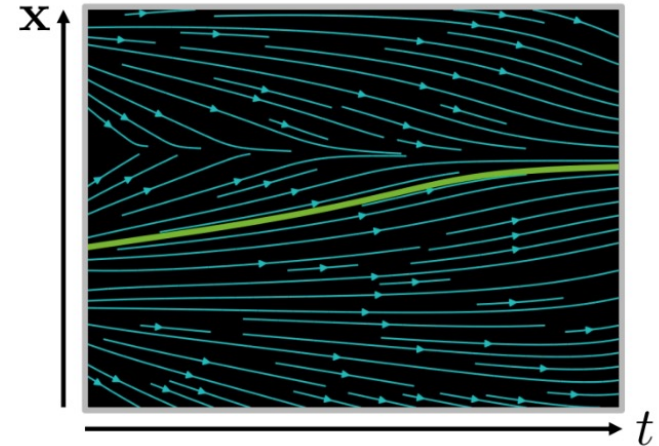
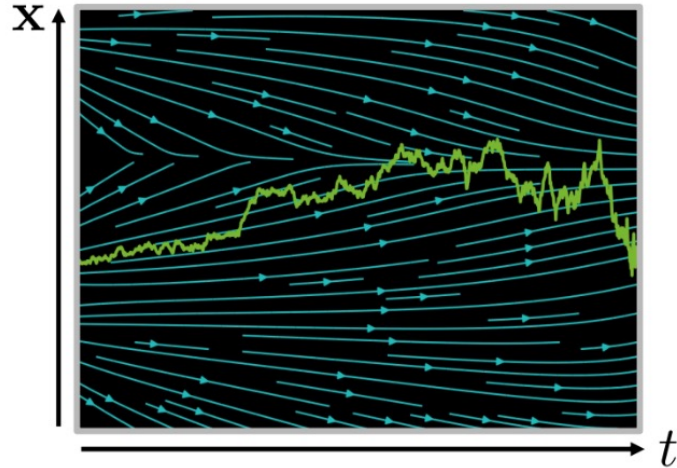
Semantic Image Interpolation with Probability Flow ODE



Continuous changes in latent space (x_T) result in continuous, semantically meaningful changes in data space (x_0) !

[Dockhorn et al., ICLR 2022](#)

Synthesis with SDE vs. ODE



- **Generative Reverse Diffusion SDE (stochastic)**

$$dx_t = -\frac{1}{2}\beta(t)[x_t + 2s_\theta(x_t, t)]dt + \sqrt{\beta(t)}d\bar{\omega}_t$$

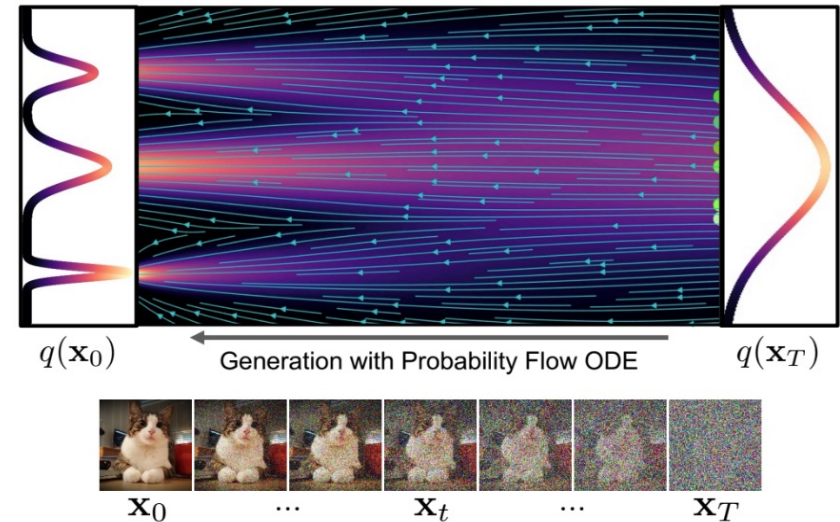
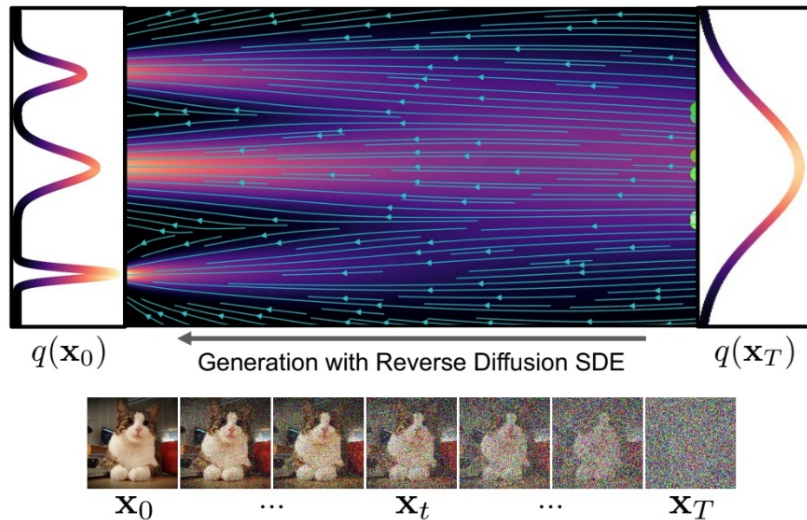
- **Generative Probability Flow ODE (deterministic)**

$$dx_t = -\frac{1}{2}\beta(t)[x_t + s_\theta(x_t, t)]dt$$

[Song et al., “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021](#)

Sampling from “Continuous-Time” Diffusion Models

How to solve the generative SDE or ODE in practice?



Generative Diffusion SDE:

$$d\mathbf{x}_t = -\frac{1}{2}\beta(t)[\mathbf{x}_t + 2s_\theta(\mathbf{x}_t, t)]dt + \sqrt{\beta(t)}d\bar{\omega}_t$$

➡ *Euler-Maruyama:*

$$\mathbf{x}_{t-1} = \mathbf{x}_t + \frac{1}{2}\beta(t)[\mathbf{x}_t + 2s_\theta(\mathbf{x}_t, t)]\Delta t + \sqrt{\beta(t)\Delta t}\mathcal{N}(0, I)$$

Probability Flow ODE:

$$d\mathbf{x}_t = -\frac{1}{2}\beta(t)[\mathbf{x}_t + s_\theta(\mathbf{x}_t, t)]dt$$

➡ *Euler's method:*

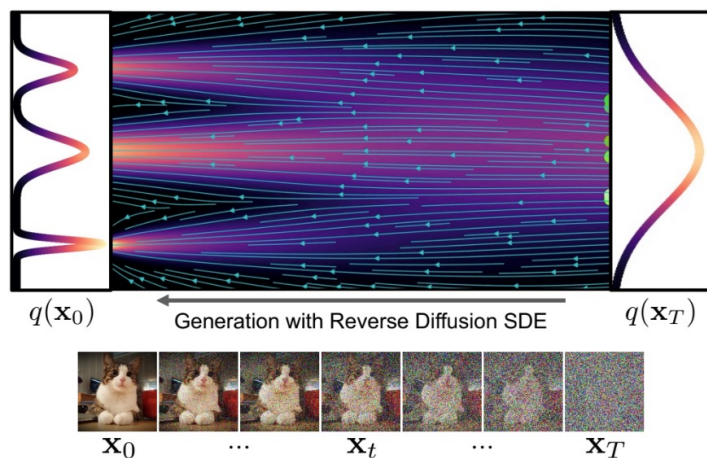
$$\mathbf{x}_{t-1} = \mathbf{x}_t + \frac{1}{2}\beta(t)[\mathbf{x}_t + s_\theta(\mathbf{x}_t, t)]\Delta t$$

In practice: High-Order ODE solvers
(Runge-Kutta, linear multistep methods, exponential integrators, ...)

[Song et al., ICLR 2021](#)

Sampling from “Continuous-Time” Diffusion Models

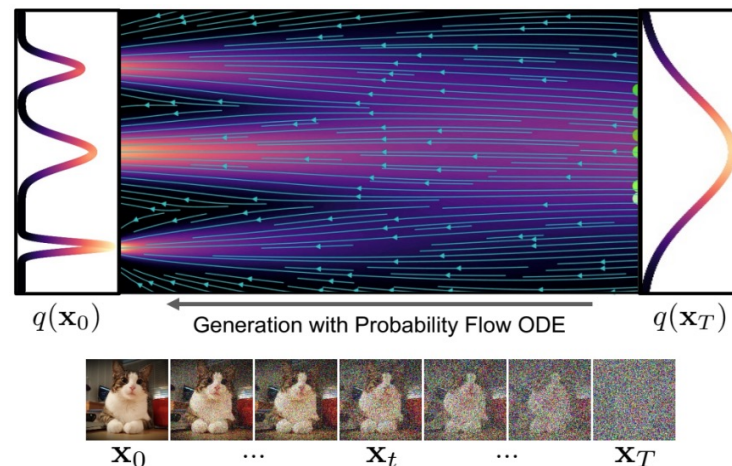
SDE vs. ODE Sampling: Pro’s and Con’s



Generative Diffusion SDE:

Pro: Continuous noise injection can help to compensate errors during diffusion process.

Con: Often slower, because the stochastic terms themselves require fine discretization during solve.



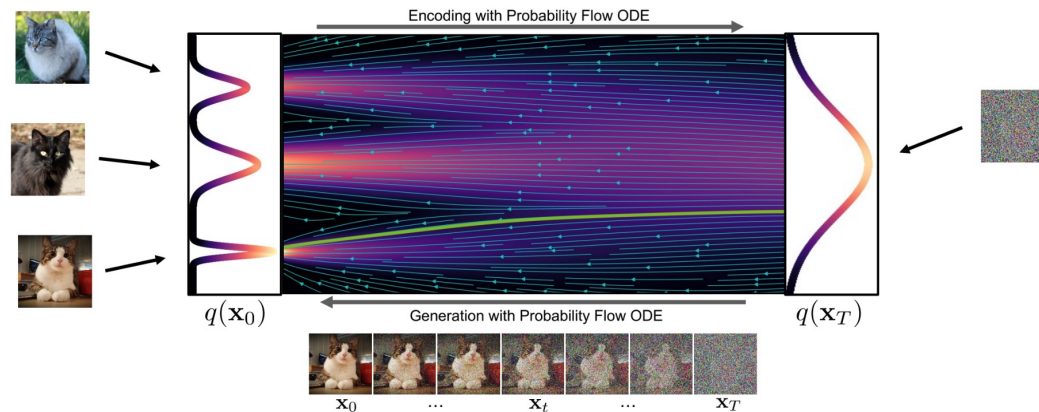
Probability Flow ODE:

Pro: Can leverage fast ODE solvers. Best when targeting very fast sampling.

Con: No “stochastic” error correction, often slightly lower performance than stochastic sampling.

[Karras et al., arxiv 2022](#)

Unique Identifiability



**Forward Diffusion
SDE:**

$$dx_t = -\frac{1}{2}\beta(t)x_t dt + \sqrt{\beta(t)}d\omega_t$$

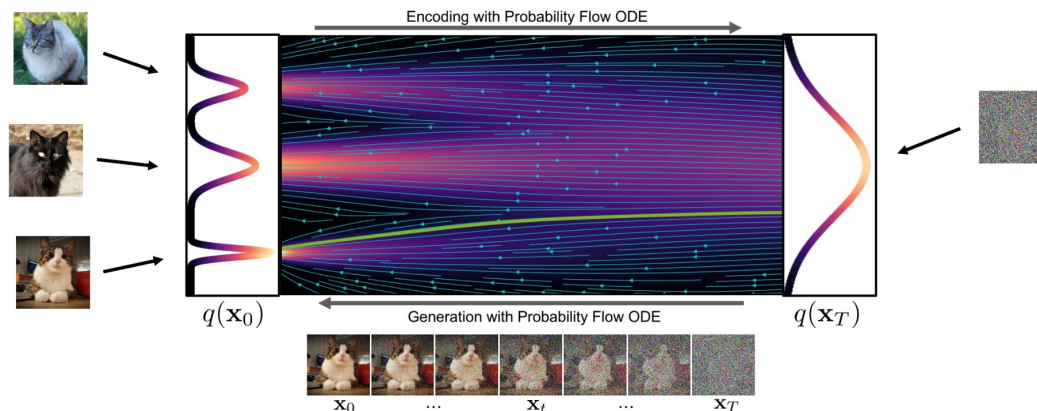
**Reverse Generative
Diffusion SDE:**

$$dx_t = -\frac{1}{2}\beta(t)[x_t + 2\nabla_{x_t} \log q_t(x_t)]dt + \sqrt{\beta(t)}d\bar{\omega}_t$$

- In diffusion models, the optimal denoising (score) function is uniquely determined by the data distribution and forward process.
- Under ideal conditions (infinite data and capacity), different models converge to the same underlying score function, leading to more stable and well-defined learning compared to GANs or VAEs.

[Song et al., ICLR 2021](#)

Why use Differential Equation Framework?



Advantages of the Differential Equation framework for Diffusion Models:

- Can leverage broad existing literature on **advanced and fast SDE and ODE solvers**
- Allows us to construct deterministic **Probability Flow ODE**
 - Deterministic Data Encodings
 - Log-likelihood Estimation
- **Clean mathematical framework** based on Diffusion Processes and Score Matching; connections to Neural ODEs, Continuous Normalizing Flows and Energy-based Models

Transport View of Diffusion Models

Flow Matching

Why move beyond the score SDE framework?

- *The SDE framework is powerful but has limitations:*
 - Curved ODE trajectories: VP/VE paths are not straight => many solver steps
 - Fixed noise schedule: $\beta(t)$ or $\sigma(t)$ must be designed carefully
 - Training through the score: indirect --- learn score, then derive velocity
- *New perspective: Optimal Transport*
 - What's the most efficient way to move mass from noise to data?
 - Answer: straight lines! (in optimal transport scene)
- *Key question: Can we directly learn an ODE with straight paths?*
 - Flow matching: yes, with a simulation-free training objective!

Velocity Field

A velocity field defines how each point moves over time:

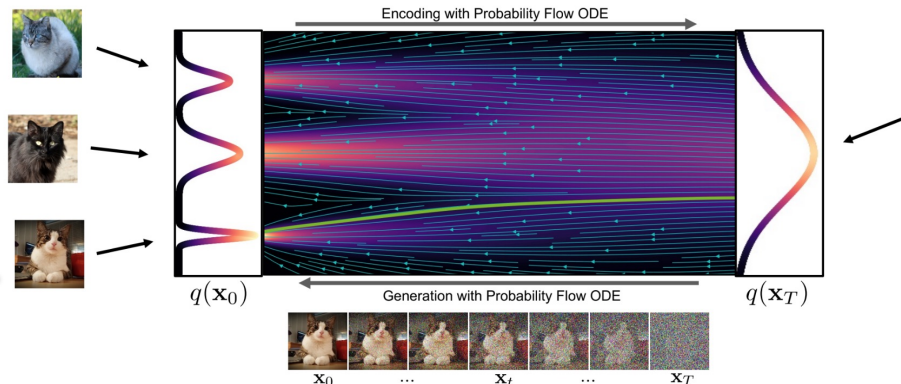
$$\frac{dx}{dt} = v(x, t)$$

$v(x, t)$: velocity at location x and time t

Defines a **continuous transformation** of samples

velocity field in Probability Flow ODE

$$dx_t = -\frac{1}{2}\beta(t)[x_t + \nabla_{x_t} \log q_t(x_t)]dt$$



velocity field defines a **deterministic generative process**

specifies: trajectories, distribution evolution, likelihood

Flow Matching: Learning the Velocity Field Directly

Goal

Learn a **velocity** field $v_\theta(x, t)$ such that:

$$p_0(x) \rightarrow p_{data}(x)$$

without SED or score estimation

Train neural network $v_\theta(x, t)$ to match:

$$E_{x,t} [\| v_\theta(x_t, t) - \text{target} \|^2]$$

x_0

x_t

x_1

$$\parallel \\ (1-t)x_0 + tx_1$$

$$\text{target} = x_1 - x_0$$

Flow Matching: Basic Overview

Once we solve the remaining issues with the flow matching objective, we train a NN model $v_t(x; \theta)$ by predicting (matching) the vector field $u_t(x)$. Usually, the UNet architecture with skip connections is used to model v_t .

Training

During training, we minimize an objective corresponding to flow matching

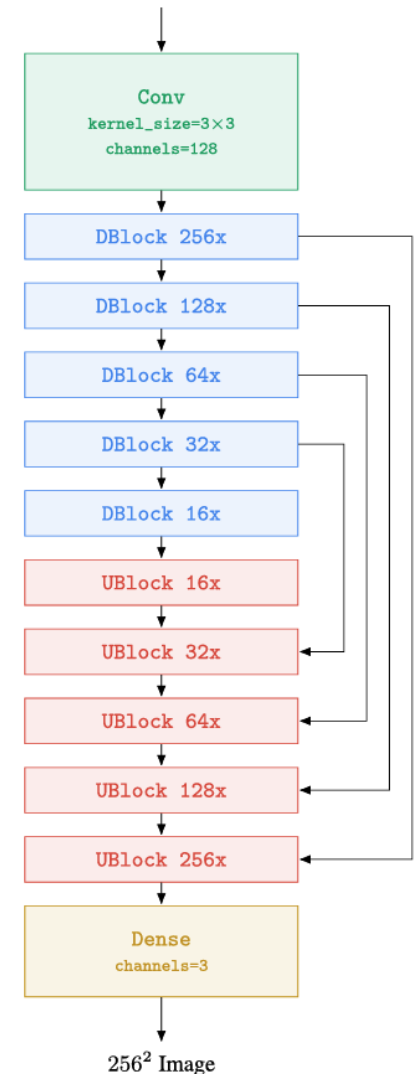
$$\mathcal{L}_{\mathcal{FM}}(\theta) = E_{t, p_t(x)} ||v_t(x; \theta) - u_t(x)||^2.$$

i.e., by performing a regression on the vector predicted by the model v_t .

Sampling

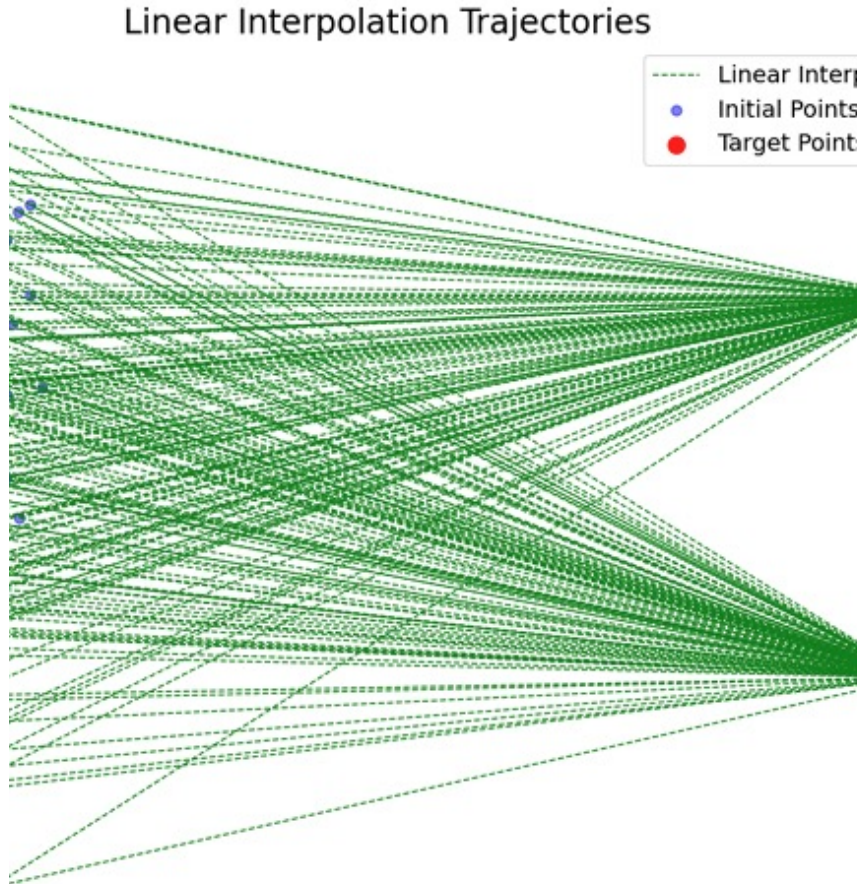
In order to generate an image, we start by sampling x_0 , and then perform numerical integration by the Euler method using steps by

$$x_{t+\frac{1}{T}} \leftarrow x_t + \frac{1}{T} v_t(x_t; \theta).$$



<https://arxiv.org/abs/2205.11487>

Rectified Flow: Motivation



- Even with flow matching and OT paths, there's a subtle problem.
- The conditional paths are straight, but the marginal paths may not be!
- Each (x_0, x_1) pair defines a straight line
- But different pairs' lines can CROSS each other
- At crossing points, the marginal velocity is multi-valued $\Rightarrow$ harder to learn
- Why crossing is bad:
- The network must output the average of multiple velocities at crossings
- This averaging blurs the velocity field
- Curved paths
- Rectified Flow: a method to systematically straighten the paths

Straightness = Fast

- **Coupling:** Sample from a noise-data pair (X_0, X_1) .
- **Interpolation:** Construct interpolation:

$$X_t = tX_1 + (1 - t)X_0$$

- **Causalization:** Convert interpolation to a causal process:

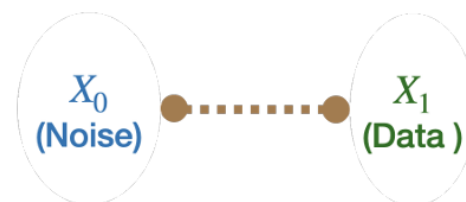
$$\dot{Z}_t = v_t(Z_t)$$

by minimizing:

$$\min_v \int_0^1 E_{(X_0, X_1)} [||\dot{X}_t - v_t(X_t)||^2] dt$$

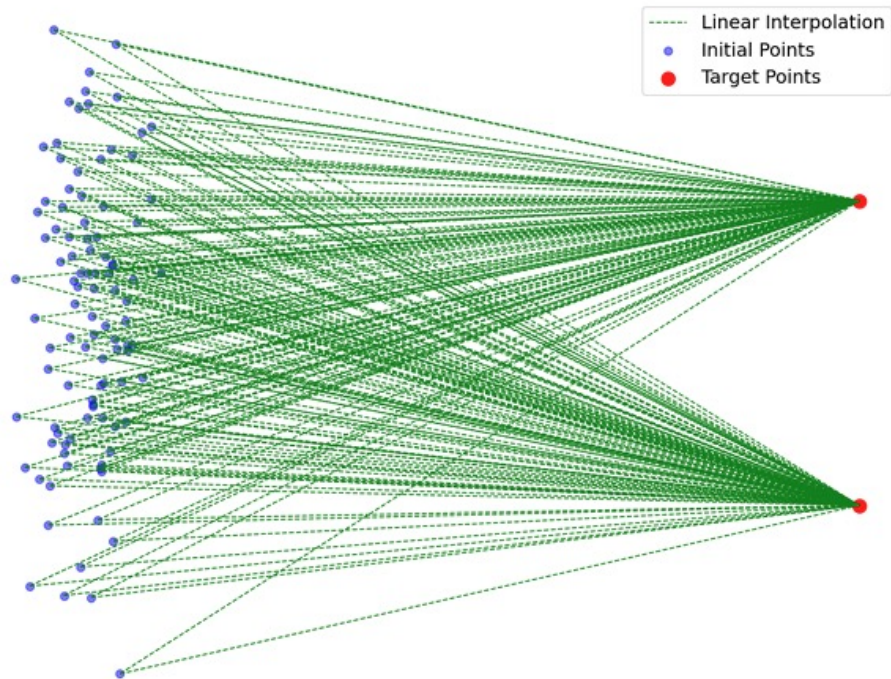
where $\dot{X}_t = X_1 - X_0$ are the line directions.

- **Reflow:** Use learned v_t to simulate the ODE from X_0 and obtain a new endpoint X_1^{new} . Then construct new couplings (X_0, X_1^{new}) .
- **Repeat.**

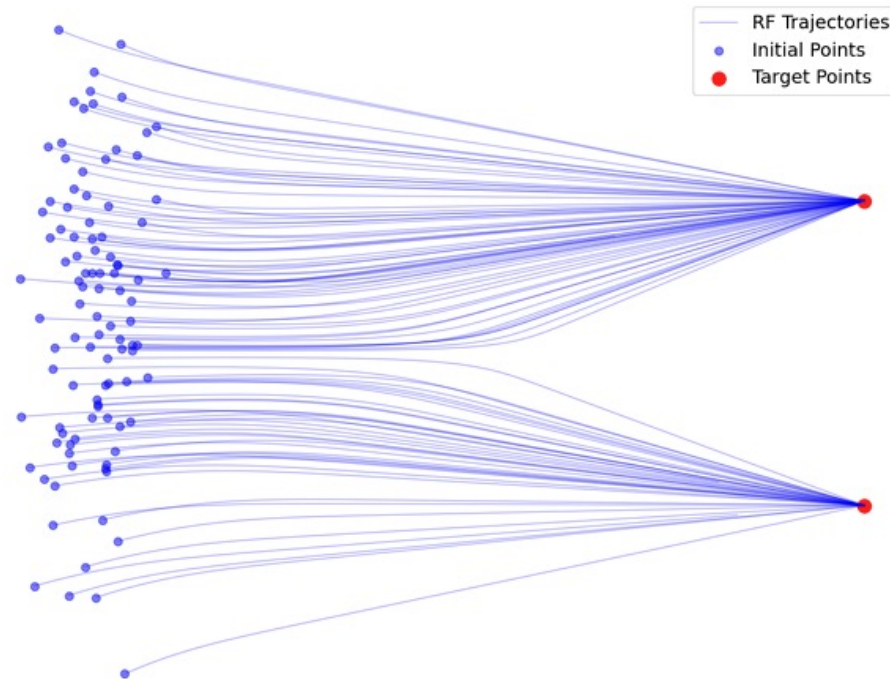


$$v^0 \rightarrow \text{reflow} \rightarrow v^1 \rightarrow \text{reflow} \rightarrow v^2 \rightarrow \dots \rightarrow v^k$$

Linear Interpolation Trajectories



Rectified Flow Trajectories



Phase 3 Summary: Transport View

- Flow Matching: simulation-free training of continuous normalizing flows
 - Conditional FM: regress $v_t(x; \theta)$ against target velocity
 - OT path: $X_t = tX_1 + (1 - t)X_0 \Rightarrow$ target velocity $= X_1 - X_0$
 - Generalizes diffusion (VP path is special case of FM)
 - Straighter paths $\Rightarrow$ fewer ODE steps than diffusion
- Rectified Flow: straighten paths via iterative Reflow
 - Reflow: use trained ODE to create causally coupled pairs
 - Retrain on coupled pairs $\Rightarrow$ non-crossing, straighter paths
 - Enables 1-step generation (without optimal distillation)

Thanks!